

Certified but Compromised: Breaking and Fixing Lightweight Proofs of Inference

Anonymous Authors

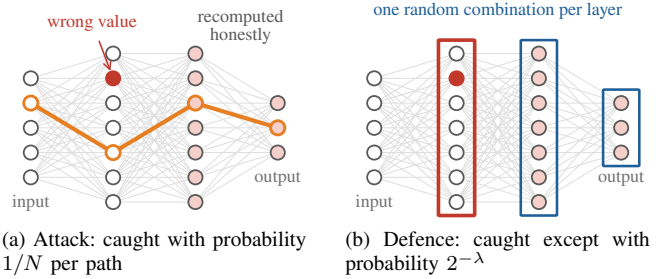
Submission under double-blind review

Abstract—Lightweight proofs of inference avoid the cost of zkSNARKs by checking only a sample of a committed execution trace. We show that such sampling is not fully sound. A provider that overwrites one neuron and recomputes the later layers honestly leaves one inconsistent site. The path protocol of Anchuri et al. accepts it with probability $1 - 1/N$ per path for a layer of width N , and on OPT-6.7B one edited neuron changed the next token for all 40 prompts tried. We prove that no sampling rule closes this gap: for any verifier that makes q queries per layer to a commitment of locality d , some network admits such a forgery that passes with probability at least $1 - qd/N$, while commitments that mix all rows reach $2^{-\lambda}$ with $O(\lambda)$ queries. We then check every weight operation of an exact int8 model with Freivalds’ algorithm against a Reed–Solomon and Merkle commitment, using only a hash function and a linear code. The protocol tolerates an untrusted commitment, proves greedy and sampled generation, and halves the proof at 2,048 tokens with a logUp-GKR. Its prover is 46 to 32,000 times faster than published zkSNARK provers. Its main cost is a proof that grows with the prompt.

1. Introduction

A company that pays a cloud provider to serve a model to its users cannot tell from a response which computation produced it. The provider may serve a cheaper model, or it may alter the computation on chosen queries to return answers of its choice, which harms any process that acts on those answers, such as payment approval or fraud flagging. The company cannot re-run the model to find out, since it lacks the weights, the hardware or both; that is why it outsourced inference. A *proof of inference* closes this gap. With each answer the provider sends a proof, and the client runs a check, much cheaper than inference, that fails with high probability unless the answer was computed by the agreed model on this input.

Proofs built on zkSNARKs [1]–[7] give this guarantee with small proofs and can hide the weights, but proving costs far more than inference: zkLLM needs about ten minutes on an A100 to prove one 2,048-token prompt to Llama-2-7B [4]. Trusted execution environments [8]–[10] are fast but place the trust in the hardware vendor. This cost has motivated lighter protocols that check only a sample of the computation. In the path protocol of Anchuri et al. [11], the provider commits to the full execution trace with a Merkle



(a) Attack: caught with probability $1/N$ per path (b) Defence: caught except with probability $2^{-\lambda}$

Figure 1. One tampered neuron whose later layers are recomputed honestly slips past a random path (a) but not past a check of every layer (b)

tree, and the client re-checks the computation along one random path from an output neuron back to the input, against a commitment to the weights. Proving drops from minutes to milliseconds. Other recent designs sample chunks of the trace [12], layers [13], [14], layers at an audited token [15] or whole requests [16], [17]. The common argument is that a different model produces a trace that differs from the correct one in many places, so a sample is likely to hit one of them. Anchuri et al. prove exactly this *other-model* soundness. They note that one path catches a single changed node with probability at most $1/N$ in a layer of width N , and they leave *full* soundness, against any wrong answer, open for large models.

The attack. A provider that wants to change an answer does not need a different model. It runs the committed model, overwrites the value of a single neuron, and recomputes every later layer honestly with the true weights. The resulting trace is inconsistent at exactly one site, which a random path visits with probability $1/N$ (Figure 1). On an MNIST network the tampered answer is accepted with probability 99.6–99.8%, and on the real OPT-6.7B, editing one feed-forward neuron changes the next token for 40 of 40 prompts. Applied only to inputs that carry a trigger, the edit is a backdoor in the serving code: the committed weights stay clean, and clean queries are answered bit for bit as before. The attack also refutes a claim made for binary classifiers, for which Anchuri et al. state that their analysis extends to full soundness. On a dog-vs-cat ResNet-18, one changed penultimate neuron flips every test query and passes a path with probability 511/512. More paths do not help at a useful cost: catching the attack on Llama-2-7B with

probability $1 - 2^{-40}$ takes about 305,000 paths, which open about 13.7 GB for a 64-token prompt.

A dichotomy for spot-checking. The attack is an instance of a general limit. Because the provider recomputes everything after the edit, the forged trace is the *honest* trace of a model that differs from the committed one in a single weight row. Suppose a verifier reads the weights through a commitment in which each symbol it can query depends on at most d rows, and makes q queries to a layer in expectation. We show that, when the other rows carry no information about where the edit is, the verifier rejects the forgery at most as often as it could guess the edited site in qd tries (Theorem 1). This holds for adaptive, value-aware verifiers that read the whole claimed trace. In the worst case no verifier beats uniform sampling: for every such verifier there is a ReLU network with a hidden layer of width N and a single-neuron forgery, with every value in its natural range, that is accepted with probability at least $1 - qd/N$. Soundness $1 - \varepsilon$ therefore requires $qd \geq (1 - \varepsilon)N$. Conversely, symbols that mix every row of a layer, such as random linear combinations of its rows or the columns of a Reed–Solomon encoding of its weights, reach $2^{-\lambda}$ with $O(\lambda)$ queries per layer, so the threshold is $\Theta(N)$ up to a factor $O(\lambda)$ (Theorem 2). That spot checks of an unencoded object catch a one-symbol change only with probability about q/N is folklore [18]–[20]. Our contribution is the reduction that extends this limit to verifiers that read the whole trace, and its consequence: the object that must be encoded is the model, not the trace. We apply the theorem to published schemes under their own parameters (Table 1). Most state probabilistic guarantees that their sampling meets; for example, one audit of SLP’s 70B configuration accepts an edited chunk with probability 98.14%. We do not call these schemes broken. Two claims exceed what the sampling gives: the path protocol’s full soundness for binary classifiers, and the negligible soundness that TensorCommitments sets as its goal while choosing the checked layers by a rule that depends only on the weights, so that an edit in a layer it does not choose always passes. We disclosed these findings to the authors of the schemes before submission [TODO: true only once the notices scheduled for 15–19 Oct are sent; otherwise delete this sentence; summarise the replies in the disclosure paragraph of Section 3.6].

A defence that checks everything. Since no verifier has a better worst-case guarantee than uniform sampling, ours checks every weight operation of the model. The committed model is an int8 network whose results are bit-identical on the provider’s GPU and on the client. With the output, the provider sends the result of every weight operation. The verifier recomputes every operation without weights, attention included, from these values, and checks each weight operation with a few random linear combinations of its output rows (Freivalds’ algorithm [21]). A client that does not hold the weights checks each combination against a Reed–Solomon-encoded, Merkle-committed copy of them, in the style of Ligero [22], and stores only a 32-byte root per weight matrix. A tampered value anywhere in the network is then caught except with probability $2^{-\lambda}$, and the protocol

needs only a hash function and a linear code: no SNARK, no trusted hardware and no trusted setup.

This design leaves four gaps, which we close. First, its soundness theorem assumes an honestly computed commitment, and this assumption carries weight. A commitment that binds one model on half of the encoded positions and a modified model on the other half lets the modified model pass with probability about 2^{-t} per query for t opened columns, which is 2^{-35} for the smallest tree of Llama-2-7B. A one-time public setup proof, a proximity test that any client checks once, removes the assumption (Section 4.5); for open-weight models anyone can instead recompute the commitment from the checkpoint. Second, the Fiat–Shamir transcript binds the whole computation, including every constant of the operations without weights, which avoids the weak Fiat–Shamir pattern [23], [24]. Third, a generated response is proved as one prefill. The verifier checks each generated token against the claimed logits before it, either as the greedy choice or as the choice of an exact integer Gumbel-max sampler keyed by a seed that the client picks, and neither rule adds a soundness term (Section 6). Fourth, we shrink the proof, which at long prompts consists mostly of the 32-bit results of the weight operations. In V1 the provider sends instead the int8 value that the next operation reads, and proves with a logUp-GKR [25], [26] that every hidden result lies in the requantisation window of the value sent. The verifier knows each operation’s input, so these window conditions are linear in the committed weights, and the GKR’s final claim reduces to the existing folded rows and column check. V1 adds no commitment (Section 5).

Costs. Beyond the forward pass, the prover’s work without V1 grows with the model but not with the input, and the prover is the protocol’s strongest axis. Computing its column openings and folds with exact int8 tensor-core products instead of float64 makes it $3.7\text{--}4.3\times$ faster on Llama-2 blocks on an RTX 2080 Ti, and it proves a 256-token GPT-2 response to a 64-token prompt in 0.29 s. On matched models it is $46\text{--}32,000\times$ faster than published zkSNARK provers [TODO: L40S: final range; the current one is from the pre-improvement L40S run]. The main cost is the proof. It carries the results of every weight operation and so grows with the prompt: 325 MB for a 64-token Llama-2-7B query and 6.6 GB at 2,048 tokens [TODO: L40S], against 183 kB for zkLLM. V1 makes it $2.0\text{--}2.3\times$ smaller at 2,048 tokens (the full OPT-1.3B: 2.07 to 0.96 GB), at the price of a GKR in the prover whose work grows with the number of results (OPT-125M at 2,048 tokens: 0.50 to 4.85 s on the RTX 2080 Ti) and a CPU verifier that is $1.6\text{--}1.9\times$ slower at that length [TODO: L40S]. The verifier recomputes attention, so its cost grows with the square of the prompt length. With eight CPU threads, a client that holds the weights verifies a 64-token Llama-2-7B query in about 1.5 s, against 3.5 s for re-executing the model in fp32, but at 2,048 tokens the two take about the same time, about 85 s [TODO: L40S/EPYC: re-measure with the native attention kernel]. Our GPU claim decoder and fused exact attention kernel make the streaming GPU verifier $2.7\times$ faster on OPT-1.3B at that length [TODO: L40S, full models].

The protocol is not zero-knowledge: the claims of $[k/T]$ queries of T tokens reveal a weight matrix with rows of length k . It therefore targets open-weight models served by an untrusted provider, the setting of CommitLLM [15] and of Gupta et al. [27]. Finally, the provider must serve the integer model. Moving SmoothQuant’s per-channel scales [28] into the gains of the integer LayerNorm brings its WikiText-2 perplexity within a factor of 1.03 of fp32 on OPT-125M and OPT-1.3B, from 1.17 and 1.11, without changing any operation that the protocol checks **[TODO: OPT-6.7B and Qwen3-4B with smoothing]**.

Contributions. We make the following contributions.

- *An attack on spot-checking proofs of inference* (Section 3): a single-site edit followed by honest recomputation, measured on image classifiers and on the real OPT-6.7B. It passes the path protocol of [11] with probability $1 - 1/N$ per path and refutes its full-soundness claim for binary classifiers.
- *A dichotomy theorem and a systematisation* (Section 3.3, Table 1): against honest recomputation, a verifier that reads the weights through a commitment of locality d needs $qd \geq (1 - \varepsilon)N$ per layer, which mixing commitments meet with $O(\lambda)$ queries. We apply it to nine published inference-verification schemes.
- *A fully sound Freivalds-based protocol that needs only a hash function and a linear code* (Sections 4–5), sound against an untrusted committer, with a Fiat–Shamir transcript bound to the whole computation, greedy and sampled generation, and V1, which halves the proof at long prompts.
- *An evaluation* (Section 7) of the attack and the defence on image classifiers and language models up to 13B parameters, with re-execution baselines, the quality of the integer models, and comparisons with published systems **[TODO: L40S]**.

2. Related Work

zkSNARK proofs of inference. A long line of work compiles neural networks into succinct proofs. For CNNs these include sum-check and GKR provers [29] such as zkCNN [1], zkPyTorch [30] and VerfCNN [31], and pairing- or Plonk-based systems such as vCNN [32], ZEN [33], ZENO [34], ZKML [2], EZKL [3] and Bionetta [35]; Mystique [36] is an interactive, VOLE-based alternative. For language models they include zkLLM [4], zkGPT [5], DeepProve [6], ZKTorch [7], Jolt Atlas [37] and the full mode of NanoZK [14]. They give computational soundness with proofs of under a kilobyte to tens of megabytes, and many of them hide the weights. Their provers are far slower than inference. zkLLM takes 620 s on an A100 for a 2,048-token Llama-2-7B prompt, with a 183 kB proof [4]; ZKTorch takes 44 min for one Llama-2-7B token [7]; DeepProve takes 34 s for a 64-token GPT-2 prompt on a 16-core CPU [6]. DeepProve and zkAgent [38] prove a generated sequence in one pass over the prompt and the output, which is

also how we prove generation. These systems prove non-linear operations with lookup arguments, such as zkLLM’s lookups and the logUp-GKR [26] that DeepProve uses. V1 borrows this machinery for one narrow purpose, to show that hidden results lie in their requantisation windows. The weight products still go through Freivalds’ check against the existing commitment, and no activation is committed. Hollow-LLM [39] observes that a valid proof over committed private weights does not establish that the provider performed the computation of a model of the declared size, since weights with a degenerate algebraic structure keep the declared shapes but are cheap to evaluate. Our protocol shares this limitation for private weights; for open weights the commitment is recomputed from the checkpoint. Our protocol gives up the zkSNARKs’ small proofs and weight privacy in exchange for a prover that is orders of magnitude faster.

Spot-checking and sampling. Anchuri et al. [11] spot-check random paths through a committed trace and prove other-model soundness through trace separation. SLP [12] commits every chunk boundary and proves a random subset of chunks with a SNARK. CommitLLM’s routine audit [15] opens 10 of 80 layers at one token of an audited response. TensorCommitments [13] and NanoZK’s audit mode [14] check a subset of layers, SPEX [16] re-executes with a fixed probability, and IMMACULATE [17] audits a fraction of requests. TOPLOC [40] and SVIP [41] fingerprint hidden states and target model, prompt or precision substitution. Verde [42], opML [43] and TAO [44] are optimistic: an answer stands unless a party that re-runs the model disputes it, so they rely on an honest party that re-runs every query. Table 1 places these designs under our theorem. SLP and CommitLLM themselves describe the edit-then-recompute strategy [12], [15], and the calculation behind it is classical. It underlies proofs of retrievability and of data possession [18], [45], it is why spot-checkers certify only closeness [19], and it is why PCPs of proximity guarantee only that their input is close to the language while holographic proofs encode their input [20], [46], [47]. Our theorem turns this calculation into a bound for adaptive, value-aware verifiers that read the whole trace; its tight case is the optimal search for a hidden object [48]. Proof-of-Learning [49], which checks the largest weight updates of a training run, is a precedent for value-dependent sampling being gamed [50], [51].

Trusted hardware. Slalom [8] checks the linear layers of a GPU computation from inside an SGX enclave with Freivalds’ algorithm, optionally with a precomputed secret challenge. TwinShield [9] and VeriAttn [10] extend enclave-based checking to attention, and VeriAttn accepts results within a tolerance. These designs are fast, but their guarantee rests on the enclave and its vendor. Our protocol needs no trusted hardware, and its exact integer semantics leaves no tolerance in which to hide a change.

Freivalds-based checks. SafetyNets [52] uses interactive proofs for networks with quadratic activations, and ZKML [2] uses Freivalds’ check for linear layers inside a SNARK. Maverick [53] checks every linear layer of a

language model with sparse Freivalds checks against a code-encoded copy of the weights that the client stores. Gupta et al. [27] run Freivalds’ check in the exponent of a group, against verification data that the model owner generates with secret randomness, for a client that outsources inference to two non-colluding servers. CommitLLM [15] checks int8 language models with secret Freivalds vectors precomputed from the weights, at 12–14% serving overhead, and audits attention without verifying it. LAMP [54] lets a verifier that holds only a commitment check one matrix product, with Freivalds’ check and Reed–Solomon proximity testing inside a Groth16 SNARK. Our column check is that of the Ligerio and Brakedown commitments [22], [55], and our setup proof is a one-time proximity test in the style of Ligerio. Among zkSNARKs, DeepProve’s hash-based configuration commits with BaseFold [56].

Where we fit. We keep the setting of Anchuri et al., a client that holds only a commitment to the weights, and replace the sampled check, which needs $(1 - \varepsilon)N$ paths per layer for soundness $1 - \varepsilon$ (Theorem 2), with a check of every weight operation. Each of the protocols above that avoids a SNARK needs more than a cryptographic commitment: Slalom an enclave, Maverick an encoded copy of the weights, CommitLLM a secret derived from them, and Gupta et al. public data that grows with every matrix and is generated with secret randomness. zkSNARKs that verify against a Merkle root, such as DeepProve with BaseFold, need no trusted setup but prove far more slowly. To our knowledge, ours is the first Freivalds-based proof of inference that is fully sound for complete transformers, attention included, whose client needs no enclave, no stored copy of the weights and no weight-derived secret. It uses no SNARK, only a hash function and a linear code, and with the setup proof it does not trust the party that computes the commitment.

3. Attacking Spot-Checking Proofs of Inference

This section shows that a proof of inference that checks a sample of an unencoded execution trace is not fully sound, and that no sampling rule can make it so.

3.1. Setting and Threat Model

A model M has a public architecture, and the client may not hold its weights. The provider acts as the prover and the client as the verifier. Before any query the client obtains a commitment C_M to the weights. For each query x the provider returns a claimed output y and a proof. We require *completeness*, meaning that honest answers are always accepted, and *full soundness* with error ε : any $y \neq M(x)$ is accepted with probability at most ε . Anchuri et al. [11] prove only *other-model soundness*, against a provider that honestly runs a different model.

Spot-checking. The provider commits to the execution trace, and the verifier checks a few of its *sites*. A site is a

unit of the computation that can be checked locally against committed data, such as a neuron and its incoming weights in the path protocol, a chunk of layers in SLP [12], or a (layer, token) cell in CommitLLM’s routine audit [15]. For each sampled site the verifier opens its inputs, its value and the weights it uses, and checks that the value follows from them. The trace carries no error-correcting encoding, so a change to one value is visible at one site only.

Adversary. The provider controls its serving code and may deviate from the protocol arbitrarily. It keeps the committed weights. It knows the verifier’s sampling rule, including rules that adapt to the claimed values, but not the verifier’s coins. It chooses which queries to tamper with, and on those queries it returns a wrong answer (untargeted) or an answer of its choice (targeted). As in [11], C_M is computed honestly; the attack does not change it.

3.2. The Path Protocol and the Single-Site Edit

In the protocol of Anchuri et al. (RandPathTest), the prover commits to every activation a_j of the forward pass with a Merkle tree [57]. The verifier chooses a random output neuron and walks back to the input, moving to a random parent at each layer. At every node j on the path it opens the parents’ activations and the weights w_{ij} from C_M , and checks $a_j = \phi(\sum_{i \in G_j} w_{ij} a_i)$ up to a floating-point tolerance, where G_j is the set of parents of j and ϕ the activation. A path opens one weight row per layer, so proofs stay small. Anchuri et al. note that one path catches a single changed node with probability at most $1/N$ in a layer of width N . As mitigations they suggest more paths, or sampling where it matters instead of uniformly.

The single-site edit. Our adversary runs the correct model on x , picks one neuron u , and replaces a_u by a value that makes the output wrong, or equal to a class of its choice. To find u , it ranks neurons by the gradient of the decision margin and bisects for the smallest edit that changes the output. It then recomputes all later layers *with the true weights*, so u is the only node whose check fails. A path visits u with probability $1/N$ in a dense layer, and one of s paths does so with probability $1 - (1 - 1/N)^s$. Applied only to inputs that carry a trigger, the edit is a backdoor [58] in the serving code: the committed weights stay clean, and clean queries are answered bit for bit as before.

Repeated queries. Anchuri et al. observe that repeated queries amplify detection. This holds against a provider that cheats on every query. Ours tampers only with the k queries it cares about, and is caught on any of them with probability at most $1 - (1 - 1/N)^k \leq k/N$.

3.3. Why Sampling Cannot Fix It

We now show that neither mitigation helps against an adversary who knows the rule, even for an adaptive verifier that reads the entire claimed trace and chooses where to look from its values.

Model. Fix one weight layer of width N , the *target layer*. Give the verifier for free the query x , all other layers, public

side information Z (such as each neuron’s natural range) and the entire claimed trace τ' ; we call $P = (x, Z, \tau')$ its *view*. Row θ_i holds the incoming weights and the bias of neuron i , whose value on x is $h_i = f_i(\theta_i)$. The verifier learns θ only through a commitment oracle. A query σ returns a symbol that depends on the rows in its *support* S_σ , and the oracle has *locality* d if $|S_\sigma| \leq d$ for every query. A row-wise commitment, as in the path protocol, has $d = 1$ as long as its openings reveal nothing about unopened rows (Remark 1). A column of the Reed–Solomon commitment of Section 4, or a random combination $\chi^\top A$ of the layer’s rows, has support $[N]$. The verifier *touches* neuron u if some query has $u \in S_\sigma$. It chooses each query from P , its coins and the earlier answers, and it must accept the honest trace of every *admissible* instance, that is, every instance consistent with Z . In the target layer, s paths make s queries of locality 1. **Honest recomputation hides the edit.** The edited trace is the *honest* trace of an admissible model that differs from the true one in row u alone. For a ReLU neuron with a bias, that row has zero weights and bias v , or -1 if $v = 0$. We call such a forgery *explainable*. Without recomputation every child of u would be inconsistent, and an adaptive verifier could follow the inconsistencies back to u .

Theorem 1 (spot-checking is guessing the location). *Let J be the neuron that the adversary edits, and suppose that the rows other than J carry no information about J beyond the view (condition (H) of Definition 2; it holds, for example, when the rows are independent and the adversary chooses J from the values h alone). Let V make, in expectation, at most q queries that meet the target layer. Then V rejects an explainable single-site forgery with probability at most*

$$G_{qd}(J | P) = \mathbb{E}_p \left[\max_{|S| \leq qd} \Pr[J \in S | P = p] \right],$$

the probability of guessing J in qd tries from the view alone. For $d = 1$ the bound is attained by a verifier that knows the adversary’s strategy and opens the q neurons most likely to have been edited.

Opening rows therefore only confirms that they are clean, and adaptivity adds nothing beyond a good guess. This is the classical result on searching for a hidden object [48], [59], with a clean opened row in the role of an empty box. What is specific to proofs of inference is that honest recomputation makes it apply to verifiers that read the whole trace. Condition (H) is needed: if one row stores the location of the critical neuron, two adaptive queries catch the edit with certainty (Remark 2). Trained networks have correlated rows, so for them we rely on the worst case of Section 3.4.

What reading the values buys. With independent rows, the posterior on the edited neuron is proportional to the adversary’s propensity to choose it, divided by the prior probability of the value written there (Corollary 1). A value-aware sampler therefore gains on edits that write improbable values. It gains nothing on edits that write common ones, provided the adversary’s choice of neuron is itself spread out. Our measurements are consistent with this but do not test it directly. The contribution-weighted sampler

detects 9.2% of the best single-neuron edits and none of the zeroing edits; for the latter the reason is mechanical, since it never visits a neuron whose claimed activation is zero (Section 3.5). **[TODO: G1e tamper locator: top-q hit rate of a locator that ranks sites from the forged trace alone, i.e. the empirical G_q of our greedy attack and of a randomised one, MNIST MLP, $q \in \{1, \dots, 64\}$]**

3.4. A Dichotomy

Soundness quantifies over every network, and on a suitable network (H) holds by construction. This gives one half of a dichotomy. Our defence gives the other half.

Theorem 2 (dichotomy). *Consider verifiers that read each layer’s weights through a commitment of locality d that reveals nothing about unopened rows, and let q be the expected number of queries that meet a layer of width N . (a) Spot-checking. For every such verifier there is a ReLU network with a hidden layer of width N , and a single-site forgery with every value in its natural range, that the verifier accepts with probability at least $1 - qd/N$, even if it knows the later weights, every neuron’s natural range and the natural distribution of each neuron of that layer. In particular, a verifier that spot-checks q of the N sites of an unencoded trace ($d = 1$) accepts with probability at least $1 - q/N$, whatever its sampler, and soundness error at most ε on every such network requires $qd \geq (1 - \varepsilon)N$. (b) Mixing. If each query mixes all rows ($d = N$), as the folded rows and code columns of Section 4 do, then under the conditions of Proposition 3, $O(\lambda + \log L)$ queries per layer give soundness error $2^{-\lambda}$ for a network of L weight layers. For $L \leq 2^\lambda$ this is $O(\lambda)$, so $qd = O(\lambda)N$ suffices.*

Proof idea. For (a), hidden neuron i copies input bit $\sigma(i)$ for a uniformly random permutation σ . Natural inputs have at most one zero, and the output is 1 exactly when every hidden neuron is 1. On a query whose zero is at ζ , the adversary sets the neuron that copies ζ to 1, which flips the output and stays in the range $\{0, 1\}$. The forged trace, the later weights, the ranges and the hidden marginals do not depend on σ , so the edited neuron is uniform given the view, and Theorem 1 bounds detection by qd/N . Random weights on inputs that are always zero keep each opening from revealing other rows. Part (b) restricts the soundness argument of our defence to one layer: a wrong claim survives a random fold with probability $1/p$ and an opened column with probability at most $(k - 1)/n$. Appendix A gives the full proofs (Propositions 1 and 3). $\square$

What the dichotomy means. Opening s distinct neurons uniformly reaches s/N on every dense network, so in the worst case nothing beats uniform sampling. Reaching 2^{-40} then takes about $27.7N$ paths per query, which is the cost the path protocol was designed to avoid. The two regimes differ by a factor $O(\lambda)$ in qd , and in both the verifier touches every row; mixing lowers the cost of each touched row from a full row of weights to $O(\lambda)$ field elements. Part (a) is the one-sided randomised query complexity of OR_N placed inside a

network, and the spot-check calculation behind it is folklore (Section 2). Our contribution is the reduction that extends it to adaptive, value-aware verifiers that read the whole trace. Because the trace is then free to read, the object that must be encoded or folded is the model, not the trace. Our defence encodes only the model, which is why its proof still grows with the trace.

3.5. Measured Attacks

We implemented the path protocol and attacked an MNIST MLP (784–512–256–10), LeNet-5, VGG-11, VGG-16 and ResNet-18 on CIFAR-10, a 224×224 dog-vs-cat ResNet-18 that stands in for the unreleased model of [11], and the real OPT-6.7B. We compute the probability of passing one path exactly, by dynamic programming over the paths.

Reproduction. Our implementation behaves as Anchuri et al. describe. On MNIST versions of four of their substitute-model settings, honest provers passed all 3,000 runs of each, and substitute models were caught in 99.8–100% of runs, including an 8-bit quantised copy that agrees with M on all 150 evaluated queries. Without the 10^{-4} tolerance, an honest prover passes only 21–37% of runs, because a layer-wide matrix product and a single-neuron dot product round differently.

MNIST. The forgeries that Anchuri et al. consider (their substitute model, Appendices E and F and Section 7.2 of [11]) corrupt hundreds of the MLP’s 778 trace nodes and are always caught. Ours leave 1–14 inconsistent nodes. One tampered neuron passes with the predicted probability $1 - 1/N$ (99.6–99.8%), and still passes 250 paths 61% of the time. A backdoor triggered by a 3×3 bright patch returns the honest output on every clean query and succeeds on every triggered query not already in the target class. It passed 99.96% of 2,300 end-to-end runs, within sampling error of the predicted 99.8%. Keeping each tampered value below its neuron’s 95th percentile takes about five neurons and still passes 99% of the time under uniform sampling.¹

Other samplers. Against an adversary who knows the rule, uniform sampling detects 0.20% of attacks, a static importance score (outgoing weight magnitude) 0.11% and gradient saliency 0.016%. The contribution-weighted sampler, which steps to parent i with probability proportional to $|w_{ij}a_i|$, detects 9.2% of the best single-neuron edits. Zeroing about 14 of the 512 neurons instead, whose most common value is zero (65% of inputs), flips the prediction and passed 2,000 of 2,000 end-to-end runs. A floor δ , sampling by $|w_{ij}|(|a_i| + \delta)$, reaches 2% at $\delta = 1$ ($10\times$ uniform); other floors give 0.9–6×. Each figure is the detection of the best attack we found, and so an upper bound on worst-case detection. **[TODO: G3 adaptive game: per-path detection of the best value-aware sampler and of the LP-optimal sampler mixture against range-respecting edits]**

1. These percentiles were computed on test queries that include the attacked ones. **[TODO: G3: repeat with ranges calibrated on the training split; report separately the plans whose edited values, and whose every claimed value, are in range]**

with held-out ranges, MNIST MLP and CNN heads, with the paths needed for 2^{-40}]

Larger models. On every CNN and every test query, changing one penultimate neuron flips the prediction. One path catches it with probability 1/84 (LeNet-5) to 1/512 (VGG, ResNet), and a change to the least-visited neuron with probability as low as 1/28 million. This includes the dog-vs-cat ResNet-18, a binary classifier, for which Anchuri et al. state that their analysis extends to full soundness. Reaching 2^{-40} takes 2,316–14,182 paths, which open the whole model and trace. In the FFN layers of Llama-2-7B (width 11,008), 2^{-40} needs at least 305,193 paths, about 13.7GB of openings with the fp16 weights of [11] at 64 tokens.

A real LLM. On the real OPT-6.7B (fp16), the same search over FFN neurons of layers 8, 16, 24 and 31 at the last position changed the next token with one neuron for all 40 prompts. A fixed-target backdoor is more constrained. Because of the final LayerNorm, a last-layer neuron can force only its *saturation token*, so the 16,384 neurons of that layer reach 6,187 of the 50,272 tokens. The backdoor succeeds for “back” (40/40 triggered prompts, clean prompts unchanged) and fails for “hacked” (0/40). Its values exceed the neurons’ natural range, which the path protocol does not check, and 2^{-40} needs at least 454,248 paths. **[TODO: L40S: next-token attack on OPT-6.7B with values kept within ranges calibrated on WikiText-2 training text (plan item G3.5), if run]** Under the path protocol, detection depends only on how many sites are edited, so Table 10 gives the paths needed for k edited neurons in two instruction-tuned models. We do not develop a method for steering their responses.

3.6. Systematisation of Spot-Checking Schemes

Table 1 applies the bound to published schemes, under each scheme’s own sampling rule and parameters. Every acceptance probability follows from the scheme’s own formulas, and we tested no deployed system. We say that a claim exceeds the sampling only where a stated guarantee is stronger than what the scheme’s own sampling gives.

What the table shows. An edit followed by honest recomputation is caught exactly when the sample covers it: s/N for paths, 3/161 for SLP’s 70B configuration, one layer in eight for CommitLLM’s routine audit, $|S|/L$ for NanoZK’s audit mode, and never outside the weight-determined layer selection of TensorCommitments. SLP, CommitLLM, NanoZK and SPEX present their guarantees as probabilistic, and SLP and CommitLLM describe the edit-then-recompute strategy themselves [12], [15]. We therefore do not call them broken. By Proposition 2, no other value-blind sample of the same size would do better against every placement of the edit. A claim exceeds the sampling in two places. The path protocol’s full soundness for binary classifiers fails on our dog-vs-cat ResNet-18. The negligible soundness error that TensorCommitments sets as its goal is not met by a deterministic layer selection that the provider can compute itself. More generally, a selection by value

TABLE 1. SPOT-CHECKING SCHEMES AGAINST ONE CHANGED VALUE FOLLOWED BY HONEST RECOMPUTATION, UNDER EACH SCHEME’S OWN SAMPLING RULE AND PARAMETERS

Scheme	What is checked	Edit accepted with	Stated guarantee	Claim beyond sampling?
Path protocol [11]	one output-to-input path; every visited neuron	$(1 - 1/N)^s$: 99.80% ($N=512$, $s=1$), 61.3% ($s=250$); 2^{-40} needs 14,182 paths (ResNet-18 ^a), and at least 305,193 and 454,248 (Llama-2-7B, OPT-6.7B ^a)	other-model soundness (proved); full soundness said to extend to binary classifiers; $1/N$ limit noted	yes, for binary classifiers ^b
SLP [12]	3 of 161 non-anchor chunks (70B)	98.14% per audit; a persistent edit needs 37 / 160 audits for 50% / 95% detection; 88.9% for TinyLlama (5 of 45)	covers proven chunks only (3/161 stated)	no
CommitLLM, routine audit [15]	1–5% of responses; one token; 10 of 80 layers (Llama-70B)	87.5% when the token is opened; $\approx 99.4\%$ per one-token response at a 5% audit rate; an edited attention output passes every tier (authors’ own finding)	exact for opened matrices; statistical coverage; attention audited, not verified	no
Tensor-Commitments [13]	layer intervals chosen deterministically from the weights	1 for an edit outside the chosen intervals; a random choice of the same size would give $1 - S /L$	soundness $\text{negl}(\lambda)$ against any PPT prover, stated as the goal	yes ^c
NanoZK [14]	full mode: every layer; audit mode: a budget S of layers	full: $\leq 3 \cdot 10^{-37}$; audit mode with a uniform 50% budget: 50%	audit mode presented as an efficiency tool, not a security claim	no
SPEX [16]	full re-execution with probability δ	$1 - \delta$ per request	confidence δ ; state sampling “cannot prevent an adversarial solver”	no
IMMACULATE [17]	a fraction of requests; verified logit statistics	a flipped step adds $1/T$ to the flagged fraction ^d	model substitution, quantisation, overbilling	no (outside claim)
TOPLOC [40], SVIP [41]	hidden-state fingerprints	no edit needed: forged tokens with honestly recomputed fingerprints pass (100%; $\geq 95\%$)	model, prompt or precision substitution; token choice out of scope	no (outside claim)
Proof-of-Learning [49]	the Q largest updates per epoch (value-aware)	100% for a “blindfolded” update [51]	probabilistic	broken before [50], [51]

Acceptance follows from each scheme’s own formulas and parameters; no deployed system was tested. ^aOur models: the dog-vs-cat ResNet-18 that stands in for the unreleased model of [11], and the real OPT-6.7B; Llama-2-7B uses its FFN width 11,008. ^bThe authors state that their other-model analysis “trivially extends to full soundness” for binary-output models, while also noting the $1/N$ limit for one changed node; their other-model theorem is unaffected, and for LLMs they leave full soundness open. ^cThe soundness definition is stated as the problem’s goal (no theorem claims it), and the gap comes from the predictable, weight-only layer selection, not from the commitment. ^dOur derivation from the flagging rule, for a response of T steps.

or by a public ranking gives an adaptive provider a place to hide. A deterministic choice is avoided with certainty, and value-dependent choices have been evaded by edits that write common values, both by our zeroing attack and by the blindfolded updates against Proof-of-Learning [51]. TOPLOC and SVIP target model substitution, and a forged answer with honestly recomputed fingerprints lies outside what they claim.

Ethics and responsible disclosure. Our attack targets verification protocols, not deployed services. Every experiment ran on our own implementations, with public datasets and open-weight models, on our own hardware. Before submission we wrote to the authors of the path protocol and of TensorCommitments, CommitLLM, NanoZK, SLP and SPEX [TODO: dates the notices were sent; scheduled for 15 Oct 2026 (Anchuri et al., TensorCommitments) and 19 Oct 2026 (CommitLLM, NanoZK, SLP, SPEX); remove SPEX here and from Table 1 if its optional notice is not sent]. For each scheme we described what we found, under which of its stated parameters, and what we do not claim, and we offered our draft and code. [TODO: summary of the replies; if none: “We had received no reply by 2 Nov

2026.”] Where a scheme states a probabilistic guarantee, we report the acceptance probability that follows from its own formulas and do not call it broken. We say that a stated guarantee is not met only for the path protocol’s full-soundness claim for binary classifiers and for the soundness goal of TensorCommitments. We release no method for steering the responses of instruction-tuned assistants.

4. The Defence: Checking Every Weight Operation

Theorem 2 leaves one way to reach soundness $2^{-\lambda}$ well below the cost of re-execution: the verifier must touch every row of every weight matrix, through checks that each mix many rows. Our defence tests random linear combinations of all the neurons of every weight operation with Freivalds’ algorithm [21]; a client without the weights checks each combination against a Reed-Solomon-encoded, Merkle-committed copy of them, as in Ligero and Brake-down [22], [55]. The provider (the prover) answers each query x with an output y and a proof and may deviate arbitrarily; full soundness requires that every $y \neq M(x)$ be

rejected except with probability ε . In mode C the client (the verifier) holds only a commitment C_M to the weights, as in the path protocol [11]. In mode K it holds the weights, and in mode K_{pre} also secret Freivalds vectors and their folded rows, precomputed once per model as in Slalom [8] and CommitLLM [15]. Appendix B contains the proofs.

4.1. The Exact Integer Model

The checks compare exact values. Floating-point results depend on the device and the summation order, and a tolerance would leave a cheating prover room to hide small changes. We therefore commit and check an integer network obtained by integer-only quantisation [60], as proof systems and enclave-based checks for inference also do [1], [2], [4], [8], [52]. Soundness refers to this integer model, not to the floating-point checkpoint; Section 6.3 measures the gap between the two.

Weight operations. A *weight operation* ℓ of the public graph computes

$$Z_\ell = A_\ell [X_\ell; 1], \quad A_\ell = [W_\ell | b_\ell], \quad (1)$$

where W_ℓ is an int8 matrix with N_ℓ rows, b_ℓ an integer bias, and X_ℓ an int8 matrix with one column for each of its T_ℓ positions (pixels or tokens). We write k_ℓ for the row length of A_ℓ . Convolutions become products through im2col, BatchNorm is folded into W_ℓ and b_ℓ , and the scales of the weights are absorbed into the integer multiplier of the next requantisation. Embedding tables are looked up.

Operations without weights. The other operations map integers to integers with public constants fixed at calibration. A requantisation with multiplier m computes $\text{clamp}(\lfloor (mz + 2^{29})/2^{30} \rfloor, -127, 127)$. The residual stream holds 23-bit integers, norms use an integer square root, activation functions and the exponential in softmax are tables of 256 entries, and softmax outputs 8-bit probabilities (Appendix B.1). The output y is the claim of the last weight operation. For a language model these are the next-token logits, which depend only on the last position, so the last block is computed there alone, except for the keys and values that attention reads. Section 6 extends the statement to generated responses.

Exact execution. Integer results do not depend on the order of summation, so prover and verifier obtain the same integers on any hardware as long as nothing is rounded. Every weight operation’s input is int8 (it comes from a clamped requantisation, table or norm), so the prover runs $W_\ell X_\ell$ on int8 tensor cores: products are at most 2^{14} in magnitude, and blocks of 2^{16} of them sum exactly in 32-bit integers. Attention runs in float32 within its exact range (Appendix B.1). A hardware error can only make an honest proof fail.

Embedding in a field. The checks run in the BabyBear field $\mathbb{F}_p$, $p = 15 \cdot 2^{27} + 1 \approx 2^{31}$. The verifier rejects every claim with $|z| \geq 2^{29}$, and setup checks that every honest claim is smaller: $128 \sum_c |w_{ic}| + |b_i| < 2^{29}$ for every row i of a product, and $\max_c |w_{ic}| + |b_i| < 2^{29}$ for an embedding. A wrong claim and the correct value then differ by less than $2^{30} < p$, so they remain distinct in $\mathbb{F}_p$.

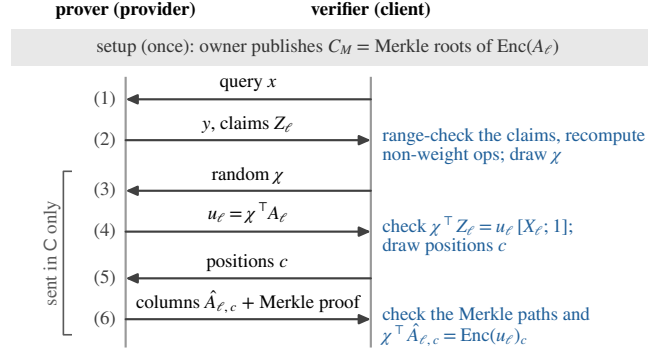


Figure 2. One query in mode C, with an honestly computed commitment. Each value is fixed before the randomness that tests it

4.2. Committing to the Weights

At setup, every row of A_ℓ , a message of length k_ℓ , is Reed–Solomon encoded [61]: the polynomial whose coefficients are the row is evaluated at the $n \geq k_\ell$ points of a multiplicative subgroup of $\mathbb{F}_p$, with n a power of two. The codewords form the rows of $\hat{A}_\ell$. Encoding is linear, so $\chi^\top \hat{A}_\ell = \text{Enc}(\chi^\top A_\ell)$, and two distinct codewords agree on at most $k_\ell - 1$ positions. A Merkle tree [57] binds the columns of $\hat{A}_\ell$; we use SHA-256 with separate domain prefixes for leaves and inner nodes. C_M consists of the roots and the public parameters of the encoding, 32 bytes per tree. It uses no secret randomness, and anyone who holds the weights can recompute it.

Commitment plans. Long codewords need fewer opened columns, and matrices whose codewords have equal length can share one tree and one Merkle multiproof. A public planning rule picks, among six candidate plans of codeword lengths and layouts, the one that sends the fewest bytes per query besides the claims, within a cap on the size of the setup; the plan is part of C_M . The five weight matrices of LeNet-5, for example, share one tree of 2^{18} positions and open 15 columns, instead of 66 in each of five trees. For a layer with many rows, such as GPT-2’s output layer ($N = 50,257$), the rows of $A_\ell^\top$ are encoded instead (the *transposed layout*), and operations that read the same input, such as the query, key and value projections, are stacked into one such matrix. An embedding table without a bias is committed as a Merkle tree over its rows, as in CommitLLM [15].

4.3. The Protocol

With y , the prover sends the *claims* Z_ℓ of every weight operation; y is itself a claim. All claims are correct if and only if three statements hold for every weight operation ℓ : (S1) X_ℓ follows from x and the earlier claims through the operations without weights; (S2) $Z_\ell = A_\ell [X_\ell; 1]$; and (S3) A_ℓ is the committed matrix. In mode C a query has three rounds (Figure 2): the claims, then the vectors χ and the folded rows, then the column positions and the opened columns. Each value is fixed before the randomness that tests it.

(S1): recomputation. The verifier computes every operation without weights itself, so each X_ℓ is its own value and the checks are chained: a prover cannot pass one operation’s check with one value and feed another into the next. Non-linear operations, softmax and the attention products need no proof.

(S2): Freivalds’ check. After all claims are fixed, the verifier draws a uniform $\chi_\ell \in \mathbb{F}_p^{N_\ell \times r}$ and checks

$$\chi_\ell^\top Z_\ell = u_\ell [X_\ell; \mathbf{1}], \quad u_\ell = \chi_\ell^\top A_\ell, \quad (2)$$

where we call u_ℓ the *folded rows*. The two sides differ by $\chi_\ell^\top D$, where $D = Z_\ell - A_\ell [X_\ell; \mathbf{1}]$. If $D_{ij} \neq 0$, then for each column χ of χ_ℓ , entry j of $\chi^\top D$ is $\chi_i D_{ij} + c$, where c does not involve χ_i , and it vanishes for one of the p values of χ_i . A wrong operation thus passes with probability at most p^{-r} , however many of its values are wrong: one tampered neuron is caught as reliably as a wrong layer. Given u_ℓ , the check costs $O(r(N_\ell + k_\ell)T_\ell)$, against $O(N_\ell k_\ell T_\ell)$ for recomputing the operation.

(S3): the column check. In modes K and K_{pre} the verifier computes u_ℓ itself. In mode C the prover sends u_ℓ . Opening a few of its entries would miss a single wrong one, as a path misses a single neuron, so the verifier tests u_ℓ through the code: it draws t distinct uniform positions c , receives the columns $\hat{A}_{\ell,c}$ with a Merkle multiproof, and checks the paths and $\chi_\ell^\top \hat{A}_{\ell,c} = \text{Enc}(u_\ell)_c$. If $u_\ell \neq \chi_\ell^\top A_\ell$, some row of $\text{Enc}(u_\ell)$ and the same row of $\chi_\ell^\top \hat{A}_\ell$ are distinct codewords, so all t columns fall where they agree with probability at most $\prod_{i < t} (k_\ell - 1 - i)/(n - i) < (k_\ell/n)^t$. A transposed matrix needs no folded row: the verifier combines positions instead of neurons, with $\chi' \in \mathbb{F}_p^{T \times r}$, and checks the encoding of $Z\chi'$ against the opened columns combined by $[X; \mathbf{1}]\chi'$ (Appendix B.2). The opened rows of a lookup table are checked against its root.

Modes K and K_{pre} . In K the verifier draws a fresh χ_ℓ for each query and computes u_ℓ itself, so the proof is the claims alone. In K_{pre} it computes the secret χ_ℓ and u_ℓ once per model. This is a designated-verifier mode: nobody else can check its proofs, and soundness is lost if χ_ℓ leaks.

Costs. The prover’s extra work, $O((r+t) \sum_\ell N_\ell k_\ell)$, does not depend on the number of positions, and no term of the verifier’s work grows with $N_\ell k_\ell T_\ell$. The proof holds the claims, which grow with T , and in mode C the folded rows, opened columns and Merkle paths, which do not. Section 5 replaces most claims by the int8 values that their consumers read, reusing the folds and the column check.

Soundness. Let $\mathcal{J}$ be the set of committed matrices: one for each weight operation in the row layout and one for each transposed matrix. Matrix j has message length k_j and codeword length n_j , and its tree opens t_j columns.

Theorem 3 (soundness). *Suppose that C_M is computed honestly from the weights of M and that the prover finds no hash collision. For every input x , which the prover may choose, and every prover, the verifier of mode C accepts an*

output $y \neq M(x)$ with probability at most

$$\varepsilon_C = \sum_{j \in \mathcal{J}} \left[p^{-r} + \prod_{i < t_j} \frac{k_j - 1 - i}{n_j - i} \right].$$

In mode K the bound is $\varepsilon_K = \sum_\ell p^{-r}$, over the weight operations that are not looked up. In mode K_{pre} , over Q queries that reuse the secret vectors, the verifier accepts some $y \neq M(x)$ with probability at most $Q \varepsilon_K$, even if the prover learns which check failed in every rejected query.

Proof sketch. The first wrong weight operation has a correct input, which the verifier computed from correct claims. With the true u_ℓ Freivalds’ check fails except with probability p^{-r} , and with another u_ℓ the column check fails except with the column term. In K_{pre} , until a wrong claim passes, every outcome is a function of the prover’s messages, so its view does not depend on χ_ℓ (Appendix B.2). $\square$

Parameters. We set $\beta = \lambda + \log_2(2|\mathcal{J}|)$ and $r = \lceil \beta / \log_2 p \rceil$, and each tree opens the least t that keeps the column term of every member below $2^{-\beta}$. Then $\varepsilon_C \leq 2^{-\lambda}$; at $\lambda = 128$, $r = 5$. Each folded row and each column mixes all rows of its matrix, so the protocol is in the second regime of Theorem 2: it touches every row, at a cost of $O(\lambda)$ field elements per row instead of k_ℓ weights.

4.4. Non-Interactive Proofs

Interactively, each challenge is expanded with SHAKE-256 [62] from a fresh 256-bit key that the verifier draws. Under Fiat–Shamir [63] the keys are derived from the transcript instead, and two details of this derivation matter for soundness.

The statement binds the computation. Challenges that do not depend on the whole statement make one transcript valid for several statements, the weak Fiat–Shamir pattern that has broken deployed proof systems [23], [24]. Our verifier recomputes the operations without weights, so their constants belong to the statement. The transcript therefore starts with a *graph digest*, the SHA-256 hash of a canonical description of the graph: every operation in order with its inputs and output, the shape, layout and input bound of each weight operation, and the function of each other operation with every constant it uses, table bytes included. It then absorbs the security parameters, the commitment, the input x and, for sampled generation, the sampler’s parameters and the client’s seed (Section 6). Two models that differ in one requantisation multiplier thus produce different challenges.

A sponge transcript. The transcript is a SHAKE-256 sponge with a protocol-specific domain string, and messages are absorbed with labels and length prefixes, so its encoding is injective. A challenge key is 32 bytes squeezed from a copy of the sponge after it absorbs the challenge’s label; rejection sampling expands it into unbiased field elements and distinct column indices. The challenges are thus random-oracle outputs on prefix-free inputs, which a Merkle–Damgård hash such as SHA-256 would give only with a separate argument [64]. The Merkle trees keep SHA-256, from which they need only collision resistance.

Soundness. The interactive protocol is round-by-round sound [65]: once a wrong claim is fixed, the vectors χ rescue the prover with probability at most p^{-r} , and once a wrong folded row is fixed, the columns rescue it with probability at most its column term. It follows [65]–[67] that a prover making Q random-oracle queries, for example to rehash altered claims in search of favourable challenges, succeeds with probability at most $Q\varepsilon_C + O(Q^2 2^{-256})$. We add 64 bits to β , which keeps $Q\varepsilon_C \leq 2^{-\lambda}$ for $Q \leq 2^{64}$ and raises r to 7 at $\lambda = 128$. The encoding of the claims is not canonical, but each re-encoding of the same claims costs one oracle query, which the bound already counts.

4.5. Untrusted Commitments

Theorem 3 assumes that C_M is honest, as the path protocol does [11]. This holds by verification when the client can recompute C_M , for instance from the published int8 checkpoint of an open-weight model. When the provider commits, the per-query column check does not enforce it.

The split-commitment attack. A committer can bind any matrix E_j under a root, not only a matrix of codewords. Let A and A' be two models that differ in matrix j , for example by a backdoor. The committer binds the encoded columns of A_j on a secret half of the positions of j 's tree and those of A'_j on the other half. A prover that computes with A' sends $u = \chi^\top A'_j$, passes Freivalds' check, and passes the column check whenever all t opened columns fall into the half of A'_j :

$$\Pr[\text{accept}] \geq \prod_{i < t} \frac{n/2 - i}{n - i} \approx 2^{-t}. \quad (3)$$

For the smallest tree of Llama-2-7B ($t = 35$) this is about 2^{-35} per query, not 2^{-128} . On a small decoder with one weight row changed in one operation, the acceptance rate over 300 queries with a small t matched Eq. (3). Against a malformed commitment a column is worth about one bit instead of $\log_2(n_j/k_j)$, so per-query checks alone would need $3.2\text{--}6.6\times$ the column bytes on the models of Table 2.

One combination over an extension field. In a 31-bit field, a random combination of the rows of a malformed matrix lands close to the code with probability up to about n/p . The protocol, however, checks r independent vectors at the same columns.

Fact 1. Let $\mathbb{K}_s = \mathbb{F}_p[\xi]/(f)$ for an irreducible f of degree s , and let $\chi_1, \dots, \chi_s \in \mathbb{F}_p^R$ be independent and uniform. Then $\psi = \sum_i \xi^{i-1} \chi_i$ is uniform in $\mathbb{K}_s^R$. For every $E \in \mathbb{F}_p^{R \times n}$ and set S of columns, the s combinations $\chi_i^\top E$ agree on S with codewords of the Reed–Solomon code over $\mathbb{F}_p$ if and only if $\psi^\top E$ agrees on S with a codeword of the Reed–Solomon code over $\mathbb{K}_s$ with the same evaluation points.

Proximity errors therefore scale with p^{-s} rather than p^{-1} . For the per-query vectors ($s = r = 5$ at $\lambda = 128$), the unique-decoding error n/p^r is at most $2^{-136.6}$ for $n \leq 2^{18}$. An untrusted commitment needs more columns, not more randomness.

A one-time setup proof. Once per model, the committer publishes with C_M a non-interactive proof that every committed matrix E_j is e_j -close to a matrix of codewords: it differs from $\text{Enc}(A_j^*)$ on at most $e_j < (n_j - k_j + 1)/2$ columns, which makes A_j^* unique. With Fiat–Shamir challenges over the graph digest and every tree's root and shape, (1) the verifier draws s uniform base-field combinations ψ_j of the rows of each matrix; (2) the committer sends $\psi_j A_j^*$; (3) the verifier draws t_j^0 distinct uniform columns of each tree, which the committer opens with a multiproof; and (4) the verifier checks the multiproofs and $\text{Enc}(\psi_j A_j^*)_c = \psi_j E_{j,c}$ (parameters in Appendix B.4). Transposed matrices need this proof: their per-query combination $[X; 1]\chi'$ lies in the span of the activations and is not uniform, so it cannot establish closeness.

Theorem 4 (soundness with an untrusted commitment). *Model the hash functions as random oracles, and let the committer and the prover make at most Q oracle queries in total. Let C_M be any string and let π be any setup proof that the verifier accepts. Then, except with probability*

$$\varepsilon_0 \leq Q \sum_{j \in \mathcal{J}} \left[\frac{n_j}{p^s} + \prod_{i < t_j^0} \frac{n_j - e_j - 1 - i}{n_j - i} \right] + O(Q^2 2^{-256}),$$

every matrix E_j that C_M binds is e_j -close to $\text{Enc}(A_j^)$ for a unique A_j^* . Let M^* be the model with weights A^* whose weight products are computed in $\mathbb{F}_p$ and lifted to $(-p/2, p/2)$. For every input x and every prover, the verifier of mode C then accepts $y \neq M^*(x)$ with probability at most*

$$\varepsilon_C^* = \sum_{j \in \mathcal{J}} \left[p^{-r} + \prod_{i < t_j} \frac{k_j + e_j - 1 - i}{n_j - i} \right],$$

or $Q\varepsilon_C^ + O(Q^2 2^{-256})$ under Fiat–Shamir. If the client recomputes C_M , then $e_j = 0$, no setup proof is needed, and the bound is that of Theorem 3.*

Proof sketch. If an extracted E_j is far, Fact 1 and correlated agreement for Reed–Solomon codes in the unique-decoding regime [68] make $\psi_j^\top E_j$ far from the code over $\mathbb{K}_s$ except with probability n_j/p^s . Online, a wrong folded row survives only on the at most $k_j - 1 + e_j$ positions where its codeword agrees with the true one or where E_j is corrupted, a set fixed before the query (Appendix B.4). $\square$

C_M thus fixes M^* before any query, and M^* is the integer model of Section 4.1 when A^* is int8 and meets the range bound. A client that sees neither the weights nor a range proof on them learns that the provider is bound to one fixed model, but not which one.

The trade-off. A larger radius e_j shortens the setup proof, since each setup column then catches a far matrix with probability about e_j/n_j , and lengthens every query, whose bound has $k_j + e_j$ in place of k_j . Each tree takes the radius, among the fractions $2^0, \dots, 2^{-10}$ of the unique-decoding radius, that minimises $t_j^0/a + t_j$ over a amortised queries (Table 2). At $a = 1,000$, Llama-2-7B pays 11.3 GB once and 8% more column bytes per query; at 2,048 tokens the change per query is below 1%. Our implementation covers

TABLE 2. UNTRUSTED COMMITMENTS AT $\lambda = 128$: THE ONE-TIME SETUP PROOF AND THE OPENED COLUMNS PER QUERY, WITH SETUP PARAMETERS AMORTISED OVER $a = 100$ OR 1,000 QUERIES (INTERACTIVE QUERIES, 4 BYTES PER FIELD ELEMENT)

Model	setup proof		columns per query (MB)	
	$a=100$	$a=1,000$	honest	untrusted, $a=100 / 1,000$
GPT-2	213 MB	686 MB	3.22	6.84 / 4.63
Llama-2-7B	2.8 GB	11.3 GB	124.7	168.5 / 134.9
OPT-6.7B	2.5 GB	10.2 GB	85.9	130.7 / 97.1
Llama-2-13B	4.0 GB	16.4 GB	149.3	217.4 / 165.9

commitment plans, including shared trees and transposed matrices, and rejects the setup proof of the split commitment. **[TODO: L40S: setup prover and verifier time for GPT-2 and the 7B models.]**

Scope. Neither the setup proof nor mode C hides the weights: the verifier knows X_ℓ and Z_ℓ , so the claims of about $\lceil k_\ell/T \rceil$ queries reveal A_ℓ . Mode C thus suits weights that the client may see. When the int8 checkpoint is public, recomputing C_M also settles which model is committed, and it is cheaper than a setup proof, which for GPT-2 is larger than the int8 weights. The setup proof serves clients that cannot recompute C_M .

5. V1: Sending Int8 Values Instead of Claims

At long prompts the proof consists mostly of claims, at 16–19 bits each after compression; at 2,048 tokens the proof for Llama-2-7B is about 6.6 GB, close to its int8 weights. In V1 the prover sends, in place of each claim, the int8 value that the next operation reads. V1 adds no commitment and halves the proof at 2,048 tokens, but costs prover time, so it is an option next to the protocol of Section 4, which we call v0.

5.1. The Observation

In a decoder, every weight operation other than the embeddings and the LM head is read by exactly one operation without weights, a requantisation or a residual addition. The query, key, value, gate and up projections (fc1 in OPT) feed a requantisation $a = \text{rq}_\ell(Z_\ell)$, where $\text{shift}_\ell(z) = \lfloor (m_\ell z + 2^{29})/2^{30} \rfloor$, $\text{rq}_\ell(z) = \text{clamp}(\text{shift}_\ell(z), -127, 127)$ and m_ℓ is a public integer multiplier. The attention output and down projections (fc2) feed a residual addition, which adds $\Delta = \text{shift}_\ell(Z_\ell)$ to the residual stream and clamps only the sum. The next operation thus reads a or Δ , never Z_ℓ , and in V1 the prover sends $s_\ell = a_\ell$ or $s_\ell = \Delta_\ell$ instead of Z_ℓ . The verifier still recomputes every operation without weights (statement (S1) of Section 4.3) and so holds every input X_ℓ , but it can no longer check Freivalds’ equation. It must instead be convinced that the hidden $Z_\ell = A_\ell \hat{X}_\ell$, with $\hat{X}_\ell = [X_\ell; 1]$, requantises to s_ℓ .

5.2. Windows and Exceptions

The integers that shift to a value v form an interval. Let $\text{lo}_\ell(v) = \lceil (v 2^{30} - 2^{29})/m_\ell \rceil$, the smallest z with $\text{shift}_\ell(z) =$

v , and $W_\ell(v) = \text{lo}_\ell(v + 1) - \text{lo}_\ell(v)$. Then

$$\text{shift}_\ell(z) = v \iff 0 \leq z - \text{lo}_\ell(v) < W_\ell(v), \quad (4)$$

and $W_\ell(v)$ is $\lfloor 2^{30}/m_\ell \rfloor$ or $\lceil 2^{30}/m_\ell \rceil$. A clamped entry, with $a = \pm 127$ and Z outside the window of a , is listed as an *exception* with its exact pre-activation z ; the verifier checks $|z| < 2^{29}$ and $\text{rq}_\ell(z) = a$ and gives the entry the window $[z, z + 1)$. Residual operations clamp only after the addition and have no exceptions. Clamps are rare: with real weights they occur at a rate of about 10^{-5} , and our measured proofs list at most 65 exceptions.

For entry (i, j) of a cut operation, let L_{ij} be $\text{lo}_\ell(s_{ij})$, or z for an exception, and W_{ij} its width; both are public. By (4), if every entry of the residue

$$\delta_\ell = A_\ell \hat{X}_\ell - L_\ell \quad (5)$$

lies in its window $[0, W_{ij})$, every sent value is the one the model computes, and the honest prover, which lists exactly the clamped entries, produces such residues. The verifier knows $\hat{X}_\ell$ and L_ℓ , so δ_ℓ is affine in the committed weights: V1 reduces to a range check on a linear image of A_ℓ .

5.3. A Windowed LogUp with Integer Multiplicities

We prove all these range memberships at once with a lookup argument based on logarithmic derivatives (logUp) [25]. Under Fiat–Shamir, the 64 extra bits of Section 4.4 require its challenges to come from an extension of BabyBear of degree at least 7. We use $\mathbb{K} = \mathbb{F}_p[\xi]/(\xi^8 - 11)$, a field because 11 is a quadratic non-residue modulo p and $p \equiv 1 \pmod{4}$, in interactive runs as well. Entry (i, j) becomes the leaf $v_{ij} = \delta_{ij} + \xi W_{ij} \in \mathbb{K}$, and the table is $\mathcal{T} = \{t + \xi w : w \in \mathcal{W}, 0 \leq t < w\}$, where $\mathcal{W}$ holds the width 1 and the two widths of each cut operation. Since 1 and ξ are linearly independent over $\mathbb{F}_p$, a leaf lies in $\mathcal{T}$ exactly when $\delta_{ij} \bmod p \in [0, W_{ij})$, so one table, of $0.9\text{--}1.5 \cdot 10^6$ elements with random weights and about $2 \cdot 10^4$ with real ones, serves every width with one leaf per entry. The prover sends a multiplicity μ_τ for each $\tau \in \mathcal{T}$, and the verifier checks

$$\sum_i \frac{1}{\alpha - v_i} = \sum_{\tau \in \mathcal{T}} \frac{\mu_\tau}{\alpha - \tau} \quad (6)$$

at a random α , the left sum running over all $N_{\text{leaf}} = \sum_{\ell \in \mathcal{C}} N_\ell T$ leaves of the cut set $\mathcal{C}$. The usual logUp lemma assumes fewer leaves than the characteristic, but our proofs hold up to $4.3 \cdot 10^9$ leaves, more than $p \approx 2.0 \cdot 10^9$, and p equal leaves cancel in characteristic p . We therefore check two conditions on integer multiplicities.

Lemma 1 (integer multiplicities). *Let $v_1, \dots, v_N \in \mathbb{K}$ and let $\mu : \mathcal{T} \rightarrow \{0, \dots, p-1\}$ satisfy $\sum_\tau \mu_\tau = N$ as integers. If $\sum_i 1/(X - v_i) = \sum_\tau \mu_\tau/(X - \tau)$ in $\mathbb{K}(X)$, then every v_i lies in $\mathcal{T}$.*

By the uniqueness of partial fractions, the number of leaves equal to each table element agrees with its multiplicity modulo p , and the number equal to any other value is

a multiple of p ; the two integer conditions then leave no room for the latter (Appendix C). Both are needed. If a multiplicity may reach p , a prover holding p copies of an off-table leaf adds p to the multiplicity of any table element, which keeps the sum and the identity; without the sum check it simply leaves those leaves out. Our tests confirm the lemma and both counterexamples by exhaustive search on $\mathbb{F}_{49} = \mathbb{F}_7[\xi]/(\xi^2 - 3)$. Rule (P5) below keeps honest multiplicities below 2^{29} .

5.4. Reducing the Lookup to the Weight Commitment

A fractional-sum GKR protocol [26], [29] reduces the left side of (6), which involves the unknown δ , to one evaluation. The cut operations are packed in graph order into instances of at most $2^{n_{\max}}$ leaves, each a matrix whose rows are its operations' stacked rows and whose columns are the T positions, padded to powers of two. The leaves are $(1, \alpha - v)$ at real positions and $(0, 1)$ at padding, and a binary tree adds sibling fractions, $(n, d) \oplus (n', d') = (nd' + n'd, dd')$, so that the root satisfies $n_0/d_0 = \sum 1/(\alpha - v)$. As in logUp-GKR, each layer is reduced to the next by a sum-check [69] of degree 3, ending at a random point $\rho = (r_{\text{col}}, r_{\text{row}})$ with claims $\hat{n}, \hat{d}$ on the leaf layer's multilinear extensions. An instance with n variables sends $4 + \sum_{k=1}^{n-1} (3k + 4)$ elements of $\mathbb{K}$ of 31 bytes each, 36.0 kB for $n = 27$.

Let $e \in \mathbb{K}^T$ hold $\text{eq}(r_{\text{col}}, j)$ for the positions $j < T$, let $\chi_\ell \in \mathbb{K}^{N_\ell}$ hold $\text{eq}(r_{\text{row}}, o_\ell + i)$ for the rows i of operation ℓ at row offset o_ℓ , and let $I(\rho)$, computable in time $O(n)$, be the multilinear extension of the indicator of the real positions. The true values are

$$\begin{aligned} \tilde{n}(\rho) &= I(\rho), \quad \tilde{d}(\rho) = \alpha I(\rho) + 1 - I(\rho) - V(\rho), \\ V(\rho) &= \sum_\ell \chi_\ell^\top (A_\ell \hat{X}_\ell - L_\ell + \xi W_\ell) e, \end{aligned} \quad (7)$$

summed over the instance's operations. The verifier computes every term except $(\chi_\ell^\top A_\ell)(\hat{X}_\ell e)$, whose first factor is a folded row as in Eq. (2) with a challenge over $\mathbb{K}$, that is, eight base-field folded rows, since A_ℓ has entries in $\mathbb{F}_p$. The prover sends $u_\ell = \chi_\ell^\top A_\ell$, the verifier uses $u_\ell \bar{x}_\ell$ with its own $\bar{x}_\ell = \hat{X}_\ell e$, and the existing column check tests u_ℓ against C_M with eight folded rows instead of r . For a transposed matrix the prover sends $y = A\bar{x}$, and the column check combines the opened columns with the coordinates of $\bar{x}$. In K_{pre} the precomputed secret vectors cannot anticipate χ_ℓ , so the prover sends $y_\ell = A_\ell \bar{x}_\ell$ for every cut operation and the verifier checks $\chi_s^\top y_\ell = u_s \bar{x}_\ell$ for each secret vector χ_s and its folded row u_s .

V1 therefore needs no new commitment, no NTT, no Merkle tree over activations and no proximity test: unlike lookup-based systems such as zkLLM and DeepProve [4], [6], it looks up values that are affine in weights already committed.

5.5. Protocol and Soundness

Cut set. The cut set $\mathcal{C}$ is a public function of the graph, the commitment plan and T , absorbed into the statement.

An operation is cut if (P1) its output is read only by a residual addition or by a requantisation whose clamp lies within $[-128, 127]$; (P2) it spans all T positions; (P3) its window widths lie in $[2, 2^{16}]$; (P4) its honest pre-activations are below 2^{29} in magnitude; and (P5) each width is shared by fewer than 2^{29} entries. A transposed matrix is cut with all its members or not at all. The other weight operations form the clear set $\mathcal{O}$ and keep v_0 : the embeddings, the LM head, the single-position operations of the pruned last block, and any operation that fails a rule.

One query. The prover sends the values s_ℓ , the clear claims, the exceptions and the multiplicities before $\alpha \in \mathbb{K} \setminus (\mathbb{F}_p + \xi\mathbb{F}_p)$ is drawn, runs the GKR protocols after it, sends the folds u_ℓ or y once the GKR points are known, and the columns are drawn last, so each message is fixed before the randomness that tests it. The verifier checks (6) as $\sum_\gamma n_0^\gamma/d_0^\gamma = \sum_\tau \mu_\tau/(\alpha - \tau)$ over the instances γ ; Appendix C lists all its checks.

Theorem 5 (soundness of V1). *Suppose that C_M is computed honestly from the weights of M and that the prover finds no hash collision. For every input x and every prover, the V1 verifier of mode C accepts an output $y \neq M(x)$ (in a generation query, an output that violates the token rule) with probability at most*

$$\begin{aligned} \varepsilon_C^{V1} &= \frac{N_{\text{leaf}} + |\mathcal{T}|}{p^8 - p^2} + \sum_\gamma \varepsilon_{\text{GKR}}(n_\gamma) \\ &\quad + \sum_{j \in \mathcal{J}} \prod_{i < t_j} \frac{k_j - 1 - i}{n_j - i} + \sum_{j \in \mathcal{J}_O} p^{-r}, \end{aligned}$$

where γ ranges over the GKR instances, n_γ is the number of variables of γ , $\varepsilon_{\text{GKR}}(n) = (1 + \sum_{k=1}^{n-1} (3k + 2))/p^8$, $\mathcal{J}$, k_j , n_j and t_j are as in Theorem 3, and $\mathcal{J}_O \subseteq \mathcal{J}$ holds the matrices of clear operations. In mode K_{pre} the last two sums are replaced by $\sum_\ell p^{-r}$ over the weight operations that are not looked up, and over Q queries that reuse the secret vectors a false acceptance occurs with probability at most Q times this bound.

Proof sketch (full proof in Appendix C). If all clear claims are correct and all residues lie in their windows, induction over the graph gives $y = M(x)$, so a first operation ℓ^* violates this, with an honest input. A clear ℓ^* is handled as in Theorem 3. For a cut ℓ^* , (P4) and the range checks give $|\delta| < 2^{30} + 2^{16} < p$, so the residue does not wrap around and its leaf is not in $\mathcal{T}$. Leaves and multiplicities precede α , so by Lemma 1 the identity fails as an identity of rational functions, whose poles lie in $\mathbb{F}_p + \xi\mathbb{F}_p$, and it fails at α except with the probability of the first term. If the logUp check passes anyway, some root is wrong, GKR soundness carries the error to the final claims, and the final check fails unless a fold u_ℓ or y is wrong, which the column check or, in K_{pre} , the secret vectors catch. $\square$

At $\lambda = 128$ the new terms are negligible: for Llama-2-13B at 2,048 tokens, the largest model we consider, the first is at most $2^{-215.3}$, and 79 instances of at most 2^{26} leaves give GKR terms of $2^{-230.7}$. They take two extra

shares of the union bound, which changes neither r nor the column counts. Under Fiat–Shamir each new round errs with probability at most $2^{-215.3}$, below the per-round error that the parameters of Section 4.4 allow, so the round-by-round argument carries over.

5.6. Implementation

V1 is implemented in Python and PyTorch for modes C and K_{pre} , interactive and under Fiat–Shamir. The verifier checks an instance’s round chains at once (50 ms for 20 variables) and computes $L_{\ell e}$, $W_{\ell e}$ and $\bar{x}_{\ell}$ as exact float64 products on 11-bit limbs. The GKR prover is written in Triton [70]: BabyBear elements in Montgomery form [71] on 32-bit lanes, an unrolled product in $\mathbb{K}$, and kernels for the leaves, the tree, the eq tables, the round sums and the folds, with the host drawing each challenge. Its transcripts are byte-identical to those of a reference prover in PyTorch (12 tests on the GPU). Factoring the current variable’s eq term out of each round polynomial saves 4 of the 20 products in $\mathbb{K}$ per pair. In the bottom layer, when rows are full ($T = 2^{n_{\text{col}}}$), the first $n_{\text{col}} - 1$ rounds bind position bits within rows whose numerators are all 1; a dedicated path reads only the real rows’ denominators, evaluates $d_L + d_R + \eta d_L d_R$ per pair, and adds the padding rows’ term $\eta(1 - \sum \text{eq})$ in closed form. On the RTX 2080 Ti an instance of 2^{25} leaves then takes 0.52 s (15.6 ns per leaf; 0.83 s in the first version). The test suite with V1 (1,492 tests) passes, including honest queries in eight configurations, a tampered trace value with honestly recomputed later layers (rejected at the final check in C and at the secret-vector check in K_{pre}), fake or omitted exceptions, multiplicity cheats and tampered GKR and fold messages. V1 also accepts the sampled generation of Section 6, since the LM head stays clear.

5.7. Cost

Section 7.4 measures V1 against v0 on the same queries (Table 7: Fiat–Shamir, random int8 weights, an RTX 2080 Ti prover, a CPU verifier on 8 threads of a Xeon Silver 4114). **Proof size.** At 2,048 tokens V1 is 2.03–2.29 $\times$ smaller on every build with more than one block; the full OPT-1.3B proof drops from 2.07 to 0.96 GB in mode C and from 2.02 to 0.91 GB in K_{pre} . The values shrink 2.2–2.3 $\times$: an int8 value costs about 7 bits and a residual increment about 12, against 16–19 bits for a claim. The parts that do not grow with the prompt grow instead: eight folded rows instead of r , y over $\mathbb{K}$ for transposed matrices, and the new multiplicities and GKR transcripts (under 1 MB in every measured build). GPT-2 at 64 tokens thus gains only 1.28 $\times$ (C) and 1.46 $\times$ (K_{pre}), and one Llama-2-13B block, the pruned last block in which only keys and values span all positions, 1.81 $\times$ in C. The byte model predicts 2.24 $\times$ (6.63 to 2.97 GB) for the full Llama-2-7B at 2,048 tokens in C [TODO: L40S: measure V1 on the full Llama-2-7B, Llama-2-13B, OPT-6.7B and Qwen3-4B at 2,048 tokens], and 1.8–2.0 $\times$ with the codec rates of real OPT-125M weights. On the smoothed real OPT-125M at 512 tokens, the measured gain is 1.68 $\times$ in C and

1.77 $\times$ in K_{pre} [TODO: real-weight V1 bytes at 2,048 tokens on the smoothed OPT graphs]; real OPT weights also have residual multipliers above 2^{29} , whose operations stay clear (P3).

Prover and verifier. With the GKR on the GPU and instances of at most 2^{24} leaves, the prover pays about 22 ns per cut claim end to end, 63–75% of its time. At 2,048 tokens it takes 3.4–4.9 s, 9.7–11.6 $\times$ the time of v0’s prover, whose int8 forward pass is fast on this card. The CPU verifier takes 1.6–1.9 $\times$ v0’s time at 2,048 tokens and 2.2–2.8 $\times$ for GPT-2 at 512 tokens, mostly for the final check’s two exact products per claim, $L_{\ell e}$ and $W_{\ell e}$. Llama-2-7B in mode C does not fit on the 11 GB card with the GKR [TODO: L40S: V1 timings with the L40S prover and the final verifier, including Llama-2-7B in mode C, the full OPT-1.3B, and the comparison with zkLLM’s prover]. V1 thus trades prover time for a proof about half as large, which pays off where moving and storing bytes costs more than seconds of GPU time.

Adversarial review. Before the measurements, four reviewers, covering the checks, the GKR and logUp algebra, Fiat–Shamir, and the cut’s semantics, attacked the code with scripts, and each finding was confirmed independently. No forgery against our models was found. Of five confirmed findings, two allowed forgeries on other models: the K_{pre} verifier never evaluated (P4), as its public graph has no weights (it now uses its own weights), and a clamp wider than int8 broke the bound on $|L|$ (now excluded by (P1)). Two concerned hygiene, folds accepted with coefficients in $[p, 2^{31})$ (an alias, not a forgery) and a missing defence-in-depth range check on cut inputs; both are now enforced. The fifth limits completeness: more than 2^{29} clamped entries cannot be encoded, as the exception bin is outside (P5). In mode C, (P4) is a property of the committed model, checked when the honest commitment is computed; with an untrusted commitment (Section 4.5) the bound must be published with the model, for v0 and V1 alike.

Limitations. Mode K and the streaming GPU verifier do not support V1 yet [TODO: GPU verifier with V1, if done before submission]. V1 assumes one requantisation multiplier per operation, so per-row weight scales (Section 6.3) need more widths, and CNNs that fuse ReLU into the requantisation are not covered. By the byte model the full Llama-2-7B proof still reaches the size of its int8 weights at about 4,500 tokens instead of about 2,050, and the sent values, like the claims, reveal information about the weights.

6. Generation and Model Quality

This section extends the protocol of Section 4 from one forward pass to a whole response, greedy or sampled, and measures the quality of the integer model that it certifies.

6.1. A Generated Response as One Prefix

Proving a response of G tokens as one query per prefix would repeat the prompt in every query. For Llama-2-7B

TABLE 3. A RESPONSE OF G TOKENS PROVED AS ONE PREFILL: 64-TOKEN PROMPT, RTX 2080 TI PROVER, XEON SILVER 4114 VERIFIER WITH 8 THREADS, INTERACTIVE, COMPACT ENCODING, RANDOM INT8 WEIGHTS, 3 QUERIES PER CELL. MB/TOKEN IS THE PROOF DIVIDED BY G .

Model, mode	G	Prover (s)	Verifier (s)	Proof (MB)	MB/token
GPT-2, C	1	0.15	0.21	14.9	14.9
	64	0.16	0.30	32.7	0.51
	256	0.29	0.59	87.3	0.34
GPT-2, K_{pre}	1	0.06	0.14	10.9	10.9
	64	0.10	0.23	28.6	0.45
	256	0.20	0.51	82.9	0.32
OPT-1.3B, C	1	0.61	0.76	96	96
	64	0.73	1.01	164	2.6
	256	1.05	2.18	369	1.44
OPT-1.3B, K_{pre}	1	0.15	0.43	60	60
	64	0.21	0.72	128	2.0
	256	0.64	1.71	332	1.30

with a prompt of $P = 512$ tokens and $G = 512$, this is about 1.3 TB of claims, against about 3.3 GB for the single prefill described here.

Reduction. The response $y_1, \dots, y_G$ to a prompt x is the greedy decoding if each y_i is the first largest logit at position $P + i - 1$ of the model run on $x \parallel y_1 \cdots y_{i-1}$. In the integer model every multiplier and lookup table is fixed at calibration, and every operation except causal attention acts on each position separately. A decoder with a KV cache therefore computes at each position exactly the integers of a causal prefill of $s = x \parallel y_1 \cdots y_{G-1}$. We prove one query on s , with $T = P + G - 1$ positions, and apply the LM head at the last G positions instead of the last one. Operations, weights and multipliers are unchanged, and the number of LM-head positions enters the graph digest that Fiat–Shamir absorbs (Section 4.4).

Token rule. Before its other checks, the verifier requires that, for each $i < G$, column i of the claimed $V \times G$ logits have its first largest entry at the next input token y_i ; the last column is the output, from which the client reads y_G . The rule costs one argmax over $V \times G$ values. The rest of the query is that of one prefill, with a single column opening per response in mode C.

Soundness. The soundness error is the ε of Theorem 3 for this graph: one term per matrix, independent of G . If some y_i is not the greedy token, the token rule passes only if the claimed logits at position $P + i - 1$ differ from the model’s. Some claim is then wrong, and the first-wrong-layer argument applies unchanged. Claims and tokens are fixed before the challenges are drawn, so the prover cannot adapt the response to them. In K_{pre} the rule can run online as claims arrive, and the reuse bound of Theorem 3 holds with Q counting the verdicts the prover sees.

Tests and cost. Tests on four small decoders (GPT-, Llama-, OPT- and Qwen-style) confirm that the logits at the last G positions equal those of the separate prefix queries, and that a wrong token, or logits forged to select it, is rejected by the token rule or by the LM head’s check. Table 3 gives the costs; the tampered query was rejected

in every cell. In mode C, a 256-token GPT-2 response takes 0.29 s to prove (1.1 ms per generated token), 0.59 s to verify and 87.3 MB, against 0.15 s, 0.21 s and 14.9 MB for one token. The opening and the folding of each matrix are paid once per response, and the proof grows only by the claims of the new positions. zkAgent, which also proves a whole transcript in one pass, reports 0.30 s per token for GPT-2, on other hardware and with different guarantees [38]. **[TODO: L40S: Llama-2-7B and Qwen3-4B with $P = 512$, $G \in \{64, 256\}$, modes C and K_{pre} , with the two tamper cells; replace Table 3 with the L40S numbers]**

6.2. Sampled Generation

Deployed models usually sample. A sampled response is not a function of the logits, so the greedy rule rejects it, and a rule that accepted any plausible token would let the provider choose the output. We fix the randomness instead: the client sends a fresh 32-byte seed with each query, and the response must be the output of a public, exact integer sampler keyed by that seed.

The sampler. It uses the Gumbel-max trick [72], [73]: sampling from $\text{softmax}(l/\tau)$ is equivalent to choosing $\text{argmax}_v(l_v/\tau + g_v)$ with independent standard Gumbel noise g_v . For claimed integer logits z with real values cz , the temperature becomes the integer weight $a = \text{round}(2^{16}c/\tau)$, and the noise of entry v is $\Gamma[u_v]$, where u_v is the v -th 16-bit word of $\text{SHAKE256}(\text{pvi}/\text{sampler}/\text{v1} \parallel \text{seed} \parallel \text{position})$ and

$$\Gamma[i] = \text{round}\left(2^{16} \cdot \left(-\ln\left(-\ln\frac{i+1/2}{2^{16}}\right)\right)\right), \quad 0 \leq i < 2^{16}. \quad (8)$$

The token is the first v that maximises $az_v + \Gamma[u_v]$ among the k largest entries of z (all entries if $k = 0$; ties in this selection go to the smaller index). Since $|z| < 2^{29}$, $a < 2^{24}$ and $|\Gamma[i]| < 2^{20}$, every score fits in a 64-bit integer, and every platform chooses the same token. The table is a public constant whose SHA-256 digest is fixed in the code; a platform whose logarithm rounds an entry differently recomputes it in exact decimal arithmetic. The token rule uses the sampler’s choice in place of the argmax , and Fiat–Shamir absorbs a , k and the seed into the statement.

Soundness, and why the seed comes from the client. The sampler adds no soundness term: the verifier recomputes each token from the claimed logits, which are checked like every other claim, so a token that is not the sampler’s choice on the true logits requires a wrong logit. The provider’s remaining lever is the seed. If it chose the seed, every seed would yield a validly proved response, and it could try seeds until the sampler returned the response it prefers. A fresh seed from the client, bound into the statement, makes the response a draw on the client’s own randomness and prevents a proof for one seed from serving for another. The proof does not show that the seed was random; a third party that must also trust the randomness would need a jointly generated seed, which we leave out of scope.

Distribution and tests. The sampler is a fixed, public approximation of sampling at temperature τ , with the uniform variables quantised to steps of 2^{-16} . **[TODO: measure the distance between this sampler’s output distribution and exact softmax sampling on real logits at typical temperatures]** Tests confirm that $k = 1$ or a very low temperature gives greedy decoding, that flat logits give near-uniform choices (each of 8 tokens chosen 380 to 620 times in 4,000 positions), that sampled responses are accepted in modes C and K_{pre} and with V1 (Section 5), and that the same tokens are rejected under another seed unless it happens to choose them. Stop rules, such as an end-of-sequence token or a length limit, need no proof beyond the token rule. Nucleus (top- p) sampling is not implemented. **[TODO: L40S: prover and verifier cost of a sampled response, next to the greedy cells of Table 3]**

6.3. Quality of the Certified Model

The provider must serve the exact integer model that the protocol certifies (Section 4.1): int8 weights with one scale per matrix, int8 activations with static scales, an integer LayerNorm whose int8 output is a gain times the normalised value, an exponential table with 8-bit probabilities, and a 23-bit residual stream. We load the real OPT checkpoints [74] into this graph and measure perplexity on WikiText-2 [75] in 16 windows of 128 tokens. Published results use 2,048-token sequences, so we compare ratios to fp32 on the same windows. With a scalar LayerNorm gain, perplexity is 17% above fp32 for OPT-125M and 11% above for OPT-1.3B (Table 4); in our earlier evaluation it was 33% above for OPT-6.7B (36.0 against 27.1).

Where the perplexity goes. We re-implemented the graph in floating point with each component switchable between its exact integer form and float. With every component integer it reproduces the integer graph’s logits exactly, and with every component float it matches the checkpoint’s to within $4 \cdot 10^{-5}$. Table 4 (top) gives the leave-one-out and add-one perplexities of OPT-125M. The LayerNorm outputs dominate: making them float removes more than half of the gap (75.5 to 69.4, against 64.8 in fp32). The weight matrices and the requantisation before fc1 follow, while the 8-bit softmax, the residual stream and the ReLU table each cost at most 0.1. The LayerNorm loss comes from rounding, not clipping. A few outlier channels of OPT’s residual stream, and a large channel at the beginning-of-sequence token, inflate the standard deviation, so a scalar gain that keeps them within int8 leaves the other channels few levels. The best scalar gains fall with the width (4, 3 and 2 for OPT-125M, OPT-1.3B and OPT-6.7B), which is consistent with the larger loss of the largest model.

SmoothQuant through the norm’s gains. The integer LayerNorm already multiplies the normalised value by a vector of public per-channel gains, so it can absorb SmoothQuant’s scaling [28]. Channel j gets the gain $\gamma_j = \kappa/s_j$ with $s_j = \rho_j^\alpha/\omega_j^{1-\alpha}$, where ρ_j is the 99.9th percentile of the channel’s normalised magnitude on the calibration window, ω_j is the largest magnitude in column j

TABLE 4. INTEGER OPT MODELS ON WIKITEXT-2 (16 WINDOWS OF 128 TOKENS). TOP: PERPLEXITY OF OPT-125M WITH ONE COMPONENT IN FLOAT (LEAVE-ONE-OUT) OR ONLY THAT COMPONENT INTEGER (ADD-ONE), FOR SELECTED COMPONENTS; THE INTEGER GRAPH HAS 75.5 AND FP32 HAS 64.8. BOTTOM: PERPLEXITY WITH THE BEST SCALAR NORM GAIN AND WITH SMOOTHED PER-CHANNEL GAINS ($\alpha = 0.55$), RATIO TO FP32, AND TOP-1 AGREEMENT WITH FP32 BEFORE AND AFTER SMOOTHING.

Component of OPT-125M	Leave-one-out	Add-one
LayerNorm outputs (all)	69.4	70.3
before attention	71.0	68.7
before the MLP	74.9	65.9
All weight matrices	72.5	66.6
fc1 requantisation	72.7	66.3
Softmax (exp table, 8-bit prob.)	75.9	64.9
Residual stream	75.9	64.8
ReLU table	75.5	64.8

Model	fp32	Scalar gain	Smoothed	Top-1 agr.
OPT-125M	64.76	75.52 ($\times 1.17$)	66.62 ($\times 1.03$)	0.72 $\rightarrow$ 0.82
OPT-1.3B	32.79	36.44 ($\times 1.11$)	33.70 ($\times 1.03$)	0.82 $\rightarrow$ 0.90

of the matrices that the norm feeds, and κ maps the largest $\rho_j\gamma_j$ to 127. The inverse $1/\gamma_j$ is folded into column j of those matrices before their per-matrix quantisation. This is SmoothQuant’s setting with per-tensor weights and static per-tensor activations (O3), inside the integer graph. Every requantisation multiplier stays a scalar and no operation or shape changes, so the protocol’s costs, the verifier and V1 are unaffected, and the gains are bound by the graph digest like every other constant; both proof formats accept a smoothed graph in our tests. With $\alpha = 0.55$, and clipping at the 99.99th percentile when calibrating the q/k/v, fc1 and attention-output requantisations, both models reach $\times 1.03$, and top-1 agreement with fp32 rises from 0.72 to 0.82 (OPT-125M) and from 0.82 to 0.90 (OPT-1.3B). Over 32 evaluation windows and three different calibration windows, smoothing alone keeps OPT-125M within $\times 1.03$ – 1.05 . Per-row power-of-two weight scales (four per matrix) reach $\times 1.00$ – 1.01 on OPT-125M, but they give a requantisation four multipliers, which V1 cannot yet handle, so we do not adopt them.

Comparison with published quantisation. SmoothQuant-O3, the published setting closest to ours, raises perplexity by 1.6% on OPT-175B and by 0.8% on OPT-IML-30B [28]. ZeroQuant-V2, with 8-bit per-token dynamic activations and 16-bit weights, reports about 1% on OPT-125M and 4% on OPT-1.3B [76]. In the evaluation of CushionCache, with group-wise weight scales, static W8A8 costs OPT-6.7B 5.4% and SmoothQuant-O3 nothing measurable [77]. Proof systems that keep 16-bit values stay under 1% [4], [5], whereas our range bound on the claims and the prover’s exact float32 products assume int8 activations (Section 4.1). SLP’s fixed-point TinyLlama is 8.5% above its float model and agrees with it on 84.8% of top-1 tokens [12]. From these results we set targets of $\times 1.06$, $\times 1.04$ and $\times 1.05$ for OPT-125M, OPT-1.3B and OPT-6.7B; the two smaller models meet theirs. Smoothed

runs of OPT-6.7B and Qwen3-4B need more memory and disk than our 2080 Ti setup provides. **[TODO: smoothed perplexity and top-1 agreement of OPT-6.7B and Qwen3-4B, as rows of Table 4] [TODO: real-weight end-to-end run with the smoothed graphs: prover, verifier and proof bytes, including V1, since real claims compress differently from random ones]**

7. Evaluation

We measure how often the path protocol accepts a single-site edit (Section 7.2), what the defence costs and what each improvement changes (Section 7.3), V1 against v0 (Section 7.4), when verifying is cheaper than re-running the model (Section 7.5), and how we compare with published systems (Section 7.6). Generation and the quality of the integer models are evaluated in Sections 6 and 6.3.

7.1. Setup

Implementation. The protocol is written in Python and PyTorch, with Triton kernels for the GPU claim decoder, the fused exact attention and V1’s GKR prover, and a C++/OpenMP attention kernel for CPU verifiers. Merkle trees use SHA-256 and the Fiat–Shamir transcript SHAKE-256; the field is BabyBear and V1’s logUp-GKR works over $\mathbb{F}_p$. We use $\lambda = 128$ unless stated otherwise. We write v0 for the protocol of Section 4 and V1 for the variant of Section 5. The prover and verifier improvements change how each party computes, not a byte that they exchange.

Hardware. The *base implementation*, without the improvements, was measured with an NVIDIA L40S (48 GB) as prover and GPU verifier and 8 threads of an AMD EPYC 9334 as CPU verifier. Each improvement, and V1, was measured on an NVIDIA RTX 2080 Ti (11 GB) with 8 threads of an Intel Xeon Silver 4114, against the previous code path (or v0) on the same queries; no improvement changed a claim, a proof or a verdict. The full models with all improvements will be measured on the L40S **[TODO: L40S: final run of every table in this section with the frozen code]**. Times are medians after a warm-up; proof sizes do not depend on the hardware.

Models. The image models are an MLP and LeNet-5 on MNIST, VGG-11, VGG-16 and ResNet-18 on CIFAR-10, and a 224×224 dog-versus-cat ResNet-18 that stands in for the unreleased classifier of Anchuri et al. [11]; int8 quantisation changed their accuracy by at most 0.4 points. The language models are GPT-2, OPT-125M/1.3B/6.7B, Llama-2-7B/13B and Qwen3-4B, with random int8 weights for costs, which depend only on the shapes except for the size of the compressed claims **[TODO: F3: real-weight end-to-end run of Qwen3-4B and OPT-6.7B]**. A model too large for the 2080 Ti is measured as b of its L blocks (b/L), the last one pruned. Real weights are used for the attack on OPT-6.7B and for model quality (Section 6.3).

7.2. The Attack and Its Detection

Path protocol. Section 3.5 reports the attack in full; we summarise it here. Probabilities of passing a path are computed exactly by dynamic programming over the paths. One tampered neuron, followed by honest recomputation, passes a path with probability $1 - 1/N$ (99.6–99.8% on the MNIST MLP), and 250 paths still accept it 61% of the time. A triggered backdoor passed 99.96% of 2,300 end-to-end runs. On every CNN and test query one penultimate neuron flips the prediction, including the binary ResNet-18 for which Anchuri et al. argue full soundness, and on the real OPT-6.7B one FFN neuron changed the next token for all 40 prompts. Of the samplers we tried, the best detects 2% of the attacks built against it on the MLP (Section 3.5). Catching one edited neuron with probability $1 - 2^{-40}$ takes 2,316–14,182 paths on the CNNs, which open the whole model and trace, and at least 305,193 paths for Llama-2-7B, about 13.7 GB at 64 tokens, where our proof is 325 MB at $\lambda = 128$.

Our verifier against the same attacks. The base implementation accepted all 3,792 honest queries and rejected all 2,512 attacks, from one changed value or neuron to perturbed weights, substitute models and forgeries that pass all checks but one: Freivalds’ check rejected 1,601, the column check 853, the Merkle check 56 and the range check 2. An adversarial review of V1 found forgeries only on two crafted models that violate conditions the verifier now checks. Without the setup proof (Section 4.5), a commitment that mixes two models passes about 2^{-t} of the queries; with it, it is rejected at setup **[TODO: L40S: outcome counts with the final code, including V1 and the setup proof]**.

7.3. Cost of the Defence

Base costs. With committed weights, the image models are proved in 3.8–22.9 ms and verified in 1.7–64.8 ms, with proofs $1.3\text{--}9.3\times$ smaller than their int8 weights. For the language models (Table 5), a 64-token prompt is proved in 76.1 ms for GPT-2 and 1.8 s for Llama-2-7B, whose 325 MB proof is about $21\times$ smaller than its int8 weights. At 2,048 tokens OPT-6.7B is proved in 7.7 s, but its proof grows to 5.5 GB and CPU verification to 46.5 s: the results of the weight operations grow with the prompt (98% of Llama-2-7B’s 6.57 GB proof), and the verifier recomputes attention over all pairs of positions. There the GPU verifier is $11\text{--}15\times$ faster than the CPU verifier.

Prover improvements. Table 6 measures each improvement alone. Opening the committed columns took 62% of Llama-2-7B’s base prover at 64 tokens (1.105 of 1.78 s), in float64. Splitting each field element into four signed bytes makes the opening and the fold exact int8 GEMMs with bit-identical results, and the prover $3.7\text{--}4.3\times$ faster on Llama-2 blocks. Int8 tensor cores also halve the forward pass’s weight products on Llama-sized layers, and the prover reuses the verifier’s fused attention kernel. The claim encoder turned out to be bound by a single-threaded copy of its output, so assembling the output on several threads

TABLE 5. LANGUAGE MODELS: COST OF PROVING THE NEXT-TOKEN LOGITS OF ONE PROMPT (INTERACTIVE, $\lambda=128$, RANDOM INT8 WEIGHTS). “BASE”: WITHOUT THE IMPROVEMENTS, ON AN NVIDIA L40S (PROVER AND GPU VERIFIER) AND 8 THREADS OF AN AMD EPYC 9334 (CPU VERIFIER). “NEW”: WITH ALL IMPROVEMENTS, ON THE L40S. THE IMPROVEMENTS LEAVE EVERY PROOF BYTE UNCHANGED (–: NOT MEASURED)

Model	Params	committed weights (C)						K _{pre}	
		prove		CPU verify	GPU verify		proof	CPU verify	
		base	new		base	new		base	proof
64-token prompts									
GPT-2	124M	76.1 ms	[TODO: L40S]	76.9 ms	79.9 ms	[TODO: L40S]	14.8 MB	61.8 ms	10.9 MB
Qwen3-4B	4.0B	1.2 s	[TODO: L40S]	712 ms	–	[TODO: L40S]	223 MB	692 ms	160 MB
Llama-2-7B	6.7B	1.8 s	[TODO: L40S]	1.0 s	431 ms	[TODO: L40S]	325 MB	698 ms	196 MB
Llama-2-13B	13.0B	2.7 s	[TODO: L40S]	1.5 s	–	[TODO: L40S]	469 MB	1.1 s	313 MB
2,048-token prompts									
OPT-125M	125M	396 ms	[TODO: L40S]	4.8 s	318 ms	[TODO: L40S]	348 MB	4.6 s	341 MB
OPT-1.3B	1.3B	3.8 s	[TODO: L40S]	27.4 s	2.6 s	[TODO: L40S]	2.1 GB	27.5 s	2.0 GB
OPT-6.7B	6.7B	7.7 s	[TODO: L40S]	46.5 s	3.5 s	[TODO: L40S]	5.5 GB	50.1 s	5.4 GB
Llama-2-7B	6.7B	–	[TODO: L40S]	–	–	[TODO: L40S]	6.6 GB	58.2 s	6.4 GB
Llama-2-13B	13.0B	17.6 s	[TODO: L40S]	–	–	[TODO: L40S]	10.5 GB	84.5 s	10.3 GB

The CPU verifier with the native attention kernel, in both modes: [TODO: L40S/EPYC: new CPU verify columns].

TABLE 6. EACH IMPROVEMENT AGAINST THE PREVIOUS CODE PATH ON THE SAME QUERIES (RTX 2080 Ti; CPU ROW: 8 THREADS OF A XEON SILVER 4114), WITH IDENTICAL CLAIMS, PROOFS AND VERDICTS. $T=2,048$ TOKENS EXCEPT FOR THE INT8 OPENING

Build (b/L blocks)	before	after	gain
<i>Opening and fold on int8 GEMMs (prover, C, $T=64$)</i>			
Llama-2-7B, 1/32	277 ms	64 ms	4.3×
Llama-2-7B, 4/32	653 ms	175 ms	3.7×
Llama-2-13B, 1/40	276 ms	63 ms	4.3×
GPT-2, 12/12	173 ms	154 ms	1.1×
<i>Weight products on int8 GEMMs (forward pass)</i>			
Llama-2-7B, 2/32	169 ms	87 ms	1.9×
<i>Threaded assembly of the encoded claims (encoder)</i>			
OPT-1.3B, 24/24	3.28 s	1.54 s	2.1×
Llama-2-7B, 2/32	0.542 s	0.238 s	2.3×
<i>Fused exact attention in the prover (forward pass)</i>			
OPT-1.3B, 12/24	0.538 s	0.332 s	1.6×
Llama-2-7B, 2/32	0.163 s	0.140 s	1.2×
<i>GPU claim decoder and fused attention (GPU verifier, K_{pre})</i>			
OPT-1.3B, 12/24	1.43 s	0.53 s	2.7×
Llama-2-7B, 2/32	0.36 s	0.18 s	2.0×
<i>Native exact attention (CPU verifier, K_{pre})</i>			
Llama-2-7B, 2/32	4.12 s	2.73 s	1.5×

gives 2.1–2.3×; a fused Triton encoder with the same bytes is slower (1.74 against 1.54 s on OPT-1.3B), a negative result [TODO: L40S: full Llama-2-7B and Llama-2-13B provers at 64 and 2,048 tokens].

Verifier improvements. The base GPU verifier spent most of its time decoding the claims on the host and passing about a dozen times over materialised $T \times T$ attention scores. Our GPU decoder validates every offset on the host before launching a kernel and rejects exactly the inputs that the host decoder rejects (tested on every truncation and on 400 random corruptions). Our fused attention kernel computes the same integers without a $T \times T$ buffer; its fp32 sums are exact because every partial sum is an integer below 2^{24} . Together they make the GPU verifier 2.0–2.7× faster at 2,048 tokens. Afterwards, copies take 32% of the CUDA time of an OPT-1.3B query (12 blocks) and int8 GEMMs

TABLE 7. V1 AGAINST V0 ON THE SAME QUERIES (FIAT–SHAMIR, RANDOM INT8 WEIGHTS). TIMES ON THE RTX 2080 Ti PROVER AND 8 THREADS OF THE XEON SILVER 4114 (–: NOT TIMED)

Build (b/L)	mode	proof (MB)			prover (s)	verifier (s)
		v0	V1	ratio	v0 / V1	v0 / V1
$T=64$						
GPT-2, 12/12	C	16.6	13.0	1.28×	—	—
	K _{pre}	11.0	7.6	1.46×	—	—
$T=512$						
GPT-2, 12/12	C	91.2	47.1	1.94×	0.25 / 1.38	0.73 / 1.64
	K _{pre}	85.5	41.5	2.06×	0.15 / 1.25	0.58 / 1.65
$T=2,048$						
OPT-125M, 12/12	C	349.6	172.3	2.03×	0.50 / 4.85	2.96 / 4.86
	K _{pre}	343.9	166.7	2.06×	0.39 / 4.52	2.55 / 4.85
OPT-1.3B, 24/24	C	2,073.9	960.8	2.16×	[TODO: L40S]	
	K _{pre}	2,023.1	910.6	2.22×	[TODO: L40S]	
Llama-2-7B, 2/32	C	269.5	126.4	2.13×	[TODO: L40S]	
	K _{pre}	254.3	111.3	2.29×	0.32 / 3.44	2.32 / 3.67
Llama-2-13B, 1/40	C	69.1	38.1	1.81×	—	—
	K _{pre}	60.8	29.6	2.05×	—	—

Times come from a run with GKR instances of at most 2^{24} leaves, whose proofs are within 0.3 MB of the sizes shown (2^{27} leaves).

4.4%: data movement dominates, including the proof upload that V1 halves. On a CPU, a native exact attention kernel gains 1.5× at 2,048 tokens [TODO: L40S/EPYC: full-model verifiers with all improvements].

Memory and untrusted commitments. [TODO: F4: peak GPU and host memory per cell]. Without the honest-commitment assumption (Section 4.5), Llama-2-7B needs a one-time setup proof of 2.8 GB (or 11.3 GB), which raises its per-query column bytes from 124.7 to 168.5 MB (or 134.9 MB), less than 1% of the proof at 2,048 tokens.

7.4. V1 Against v0

Proof size. V1 sends the int8 value (or shifted value) that the next operation reads instead of each 32-bit result, and proves with a logUp-GKR that the hidden result lies in that

TABLE 8. RE-EXECUTING THE FULL MODEL, IN SECONDS, AGAINST OUR VERIFIER. GPU: RTX 2080 Ti. CPU: 8 THREADS OF THE SAME XEON SILVER 4114 FOR ALL ROWS

	$T=64$	512	2,048
<i>GPU: integer forward pass / fp16 decoder</i>			
OPT-1.3B	—	—	0.94 / 0.18
OPT-6.7B	0.17 / 0.035	0.41 / 0.16	1.94 / 0.39
Llama-2-7B	0.16 / 0.037	0.35 / 0.18	2.08 / 0.57
Llama-2-13B	0.14 / 0.08	0.62 / 0.34	3.46 / 0.82
Qwen3-4B	0.17 / 0.03	0.45 / 0.09	1.92 / 0.37
<i>CPU: Llama-2-7B</i>			
our K_{pre} verifier	1.5	—	85
with native attention	[TODO: B5]	—	[TODO: B5]
fp32 re-execution	3.5	—	86
integer re-execution	6.8	—	196

Full models extrapolated linearly in the number of blocks from 1- and 2-block builds. The GPU integer times precede the int8 forward pass, which halves them.

value’s window (Section 5). At 2,048 tokens its proof is $2.0\text{--}2.3\times$ smaller than v0’s on every full or two-block build (Table 7); the full OPT-1.3B goes from 2.07 to 0.96 GB. V1’s own additions stay below 1 MB per proof. The gain is smaller where parts that V1 leaves unchanged weigh more: $1.81\times$ for the single Llama-2-13B block in C, whose opened columns are a larger share, and $1.28\text{--}1.46\times$ for GPT-2 at 64 tokens. With real weights a byte model predicts $1.8\text{--}2.0\times$ at 2,048 tokens, and the smoothed real OPT-125M measures $1.68\times$ (C) and $1.77\times$ (K_{pre}) at 512 tokens **[TODO: real-weight V1 bytes at 2,048 tokens on the smoothed OPT graphs]**.

Cost. The prover pays for the GKR, about one leaf per sent value (15.6 ns per leaf for a 2^{25} -leaf instance on the 2080 Ti). At 2,048 tokens V1’s prover thus takes about $10\times$ as long as v0’s fast int8 prover (OPT-125M in C: 0.50 to 4.85 s, 3.58 s of it the GKR), still seconds where the zkSNARK provers of Section 7.6 take minutes. The CPU verifier takes $1.6\text{--}1.9\times$ v0’s time at 2,048 tokens and $2.2\text{--}2.8\times$ at 512, mostly for the two exact products per value of its final check. V1 thus suits clients whose bandwidth matters more than the prover’s time **[TODO: L40S: V1 timings, including Llama-2-7B in C, which does not fit the 11 GB card with the GKR; streaming GPU verifier with V1]**.

7.5. Verification Against Re-Execution

In mode C the client holds only the commitment and cannot re-run the model; in K and K_{pre} it could. Table 8 times the integer forward pass, which is bit-identical to the claims and so a complete check, and a floating-point decoder of the same shapes, which is faster but cannot confirm an exact answer.

CPU. On 8 threads our K_{pre} verifier checks a 64-token Llama-2-7B query in about 1.5 s, against 3.5 s for fp32 and 6.8 s for integer re-execution. At 2,048 tokens it ties fp32 re-execution (85 against 86 s) and is $2.3\times$ faster than integer re-execution; recomputing the operations without

weights, mostly attention, takes 76% of its time **[TODO: L40S/EPYC: full-model CPU verifier with the native kernel against both re-executions on the same CPU]**.

GPU. On a GPU, re-execution is faster. The fp16 decoder runs Llama-2-7B at 2,048 tokens in 0.57 s on the 2080 Ti, while the base GPU verifier needed 4.95 s in K_{pre} on the faster L40S. Our verifier improvements extrapolate to about 1.1 s for OPT-1.3B in K_{pre} on the 2080 Ti, against 0.18 s for its fp16 re-execution **[TODO: L40S: re-execution and the new GPU verifier on the same GPU]**.

Where this leaves the verifier. We therefore do not claim that verification beats re-execution on a GPU. The GPU verifier suits an auditor or gateway that checks a provider without holding the weights, and a CPU client that holds them saves $2.3\text{--}4.5\times$ at short prompts and beats exact re-execution at both measured lengths. Since the claims of about $\lceil k/T \rceil$ queries of T tokens reveal a weight matrix with rows of length k , mode C suits open-weight models served by an untrusted provider. Re-running Llama-2-7B then takes its 6.7 GB of int8 weights and, to be fast, a GPU; verifying it takes a 32-byte root per matrix and a proof. At 2,048 tokens that proof (6.6 GB, 5.3 s at 10 Gb/s) is about as large as the weights, and V1 roughly halves it **[TODO: F7: transfer times at 1, 10 and 100 Gb/s per cell]**.

7.6. Comparison with Published Systems

Table 9 compares our costs with published systems on matched models, whose costs are as reported on their own hardware and checked against the sources. Our prover is faster in every row with a measured value, by $46\times$ (zkLLM, Llama-2-13B) to about $32,000\times$ (ZKML, VGG-16) against the zkSNARKs, partly because all of them but zkLLM ran on CPUs, and by $3.0\times$ against Maverick. Our verifier is slower in two measured rows, against ZKML on VGG-16 ($2.2\times$) and zkLLM on OPT-1.3B at 2,048 tokens ($2.9\times$), whose provers take minutes. Our proof is larger in most rows: $3.6\text{--}180\times$ against the other short-input proofs, $2,500\text{--}14,000\times$ against zkLLM on OPT and $36,000\text{--}56,000\times$ on Llama-2, where zkLLM’s proof stays below 200 kB. Where our proof is smaller, we are better on all three costs.

What this paper’s changes add. The improvements leave the proof sizes unchanged and will replace our timings **[TODO: L40S]**. V1 halves the proof where it is largest, from $3.6\times$ to $1.8\times$ DeepProve’s on GPT-2 with 512 tokens and from about $14,000\times$ to about $6,500\times$ zkLLM’s on OPT-1.3B. Only committing the activations and proving the operations without weights, as zkLLM does, would stop the proof from growing with the prompt. Our GPU verifier’s extrapolated 1.1 s for OPT-1.3B in K_{pre} on the 2080 Ti approaches zkLLM’s 0.90 s on an A100 **[TODO: L40S: zkLLM, and DeepProve if feasible, on our L40S]**. zkAgent reports about 0.30 s per token for GPT-2 generation [38]; Section 6 compares generation costs. CommitLLM and the other sampling-based schemes of Table 1 do not check every operation of every query, so we do not compare their costs.

TABLE 9. OUR COSTS AGAINST PUBLISHED SYSTEMS ON MATCHED MODELS, AS REPORTED ON THEIR OWN HARDWARE. OURS: BASE IMPLEMENTATION (BOLD WHERE LOWER; IN PARENTHESES, OURS LOWER $\downarrow$ OR HIGHER $\uparrow$ BY THE FACTOR SHOWN)

System	Model (T)	prover		verifier		proof		V1
		theirs	ours	theirs	ours	theirs	ours (v0)	
zkCNN ^a	LeNet-5	441 ms	5.7 ms (78$\times$ $\downarrow$)	5.8 ms	2.8 ms (2.1$\times$ $\downarrow$)	71.3 kB	63.3 kB (1.1$\times$ $\downarrow$)	–
Bionetta	LeNet-5	3.8 s	5.7 ms (664$\times$ $\downarrow$)	10.0 ms	2.8 ms (3.6$\times$ $\downarrow$)	880 B	63.3 kB (72 $\times$ $\uparrow$)	–
zkCNN ^a	VGG-16	88.3 s	20.0 ms (4,422$\times$ $\downarrow$)	59.3 ms	20.9 ms (2.8$\times$ $\downarrow$)	341 kB	2.2 MB (6.4 $\times$ $\uparrow$)	–
ZKML	VGG-16	10.6 min	20.0 ms (31,910$\times$ $\downarrow$)	9.6 ms	20.9 ms (2.2 $\times$ $\uparrow$)	12.1 kB	2.2 MB (180 $\times$ $\uparrow$)	–
zkGPT ^b	GPT-2 (64)	21.8 s	95.0 ms (229$\times$ $\downarrow$)	350 ms	96.8 ms (3.6$\times$ $\downarrow$)	101 kB	16.6 MB (164 $\times$ $\uparrow$)	13.0 MB
DeepProve ^c	GPT-2 (64)	34.2 s	95.0 ms (360$\times$ $\downarrow$)	1.4 s	96.8 ms (14$\times$ $\downarrow$)	21.7 MB	16.6 MB (1.3$\times$ $\downarrow$)	13.0 MB
DeepProve ^c	GPT-2 (512)	2.9 min	177 ms (995$\times$ $\downarrow$)	1.6 s	437 ms (3.8$\times$ $\downarrow$)	25.5 MB	91.2 MB (3.6 $\times$ $\uparrow$)	47.1 MB
ZKTorch	Llama-2-7B (1)	44.1 min	2.2 s (1,213$\times$ $\downarrow$)	1.7 min	333 ms (301$\times$ $\downarrow$)	22.9 MB	190 MB (8.3 $\times$ $\uparrow$)	–
Maverick ^d	Qwen3-4B (8)	354 ms	117 ms (3.0$\times$ $\downarrow$)	260 ms	111 ms (2.3$\times$ $\downarrow$)	36.1 MB	35.2 MB (1.03$\times$ $\downarrow$)	–
zkLLM	OPT-125M (2,048)	73.9 s	396 ms (187$\times$ $\downarrow$)	342 ms	318 ms (1.1$\times$ $\downarrow$)	141 kB	348 MB (2,467 $\times$ $\uparrow$)	172.3 MB ^e
zkLLM	OPT-1.3B (2,048)	3.7 min	3.8 s (58$\times$ $\downarrow$)	899 ms	2.6 s (2.9 $\times$ $\uparrow$)	147 kB	2.1 GB (13,979 $\times$ $\uparrow$)	960.8 MB ^e
zkLLM	Llama-2-7B (2,048)	10.3 min	[TODO: L40S]	2.36 s	[TODO: L40S]	183 kB	6.6 GB (36,000 $\times$ $\uparrow$)	[TODO: V1]
zkLLM	Llama-2-13B (2,048)	13.4 min	17.6 s (46$\times$ $\downarrow$)	3.95 s	[TODO: L40S]	188 kB	10.5 GB (56,000 $\times$ $\uparrow$)	[TODO: V1]

Ours: NVIDIA L40S prover; verifier on 8 threads of an AMD EPYC 9334, or our GPU verifier in the 2,048-token rows, as zkLLM runs on a GPU; committed weights and Fiat–Shamir, except against zkLLM, which is interactive (so are ours there), and against Maverick; to be replaced by the final L40S run with all improvements **[TODO: L40S]**. ^aVerifier on one core (ours: 8 threads). ^bNo prompt length stated (its matrix shapes imply 32 tokens); ours: 64. ^cBaseFold configuration, proving every position. ^dMaverick’s client stores an encoded copy of the weights, so ours run in K_{pre} , $\lambda=40$, one thread, without the compact encoding; Maverick draws a fresh challenge per query and its prover runs on 64 CPU threads. ^eV1 with Fiat–Shamir; v0 with Fiat–Shamir sends 349.6 MB and 2,073.9 MB.

8. Discussion and Limitations

Measurements. The base implementation was measured on an NVIDIA L40S and an AMD EPYC 9334, and each improvement and V1 on an RTX 2080 Ti and a Xeon Silver 4114, against the previous code on the same queries. The improvements change no byte of the proof, so the proof sizes stand, but the times of the frozen code on the L40S are not yet measured, and several cells of Tables 5, 7 and 9 are empty for that reason **[TODO: L40S: final run of the frozen code; fill every L40S cell]**. The 7B and 13B models were measured as one to four of their blocks on the 2080 Ti, and full-model figures for them come from the L40S run or from the byte model. Peak memory is not yet reported **[TODO: F4]**.

Real weights and model quality. The costs were measured with random int8 weights. They depend only on the shapes, except for the compressed size of the claims and of V1’s values. On the smoothed OPT-125M at 512 tokens, real activations compress less well than random ones as int8 values, and V1 shrinks the proof by 1.68 $\times$ in mode C (82.2 to 48.9 MB) and 1.77 $\times$ in K_{pre} , less than with random weights (1.94–2.06 $\times$ for GPT-2 at 512 tokens), also because seven of its residual operations have multipliers above 2²⁹ and stay clear under (P3) **[TODO: real-weight cells at 2,048 tokens, and the end-to-end run of Qwen3-4B and OPT-6.7B (F3)]**. Smoothing brings OPT-125M and OPT-1.3B within a factor of 1.03 of fp32. We have not yet run it on OPT-6.7B, whose integer model with a scalar gain is 33% above fp32 (36.0 against 27.1), or on Qwen3-4B, because both need more memory than our 2080 Ti setup provides. Soundness holds for any committed integer model; these numbers concern only how well that model serves.

V1. V1 is an option, not a replacement. It halves the

proof at 2,048 tokens but makes the prover about ten times slower on the 2080 Ti and the CPU verifier 1.6–1.9 $\times$ slower. Llama-2-7B in mode C does not fit on the 11 GB card with the GKR, so V1 has not been timed in that configuration, and the full 7B and 13B models have only the byte model’s prediction **[TODO: L40S: V1 on the full Llama-2-7B, including mode C, Llama-2-13B, OPT-6.7B and Qwen3-4B]**. Mode K and the streaming GPU verifier do not support V1 yet, and V1 assumes one requantisation multiplier per operation, which excludes the per-row weight scales that would bring OPT-125M to within $\times 1.00$ – 1.01 of fp32.

Proof size and the verifier. Even with V1, the proof grows with the prompt, because it carries one value for every output of every weight operation. For Llama-2-7B it reaches the size of the int8 weights at about 2,050 tokens without V1 and, by the byte model, at about 4,500 tokens with it. Only committing the activations and proving the operations without weights, as zkLLM does, would remove this growth; we leave it to future work. The verifier recomputes attention, so its cost grows with the square of the prompt length. A CPU client that holds the weights saves time over re-execution at 64 tokens and ties with fp32 re-execution at 2,048 tokens, and on a GPU re-execution is faster than our verifier. The case for the protocol is therefore a client that does not hold the weights or the hardware to run them, not a faster way to compute an answer.

Weights and trust. The protocol is not zero-knowledge: the claims of about $\lceil k/T \rceil$ queries reveal a weight matrix with rows of length k , so mode C targets open-weight models. For private weights it shares the limitation that Hollow-LLM points out for zkSNARKs [39]: the commitment binds the provider to one fixed model, but a client that sees neither the weights nor a range proof on them does not learn which model that is, and weights with a degenerate structure keep

the declared shapes while being cheap to evaluate. With an untrusted commitment, the bound (P4) of V1 and the claim range of v0 must be published with the model, and the time of the setup proof is not yet measured **[TODO: L40S: setup prover and verifier time]**. The exact sampler is a fixed approximation of softmax sampling whose distance to it we have not measured, the proof does not show that the client’s seed was random, and nucleus sampling is not implemented. **The attack and the theorem.** The lower bound needs condition (H), which we establish for random ensembles with independent rows and on a constructed worst-case network, not for a trained network, whose rows are correlated; on trained networks we report the detection of the best attacks we found, which bounds each sampler’s worst-case detection from above. We do not claim that the worst-case instance embeds in a transformer, since the LayerNorm before the MLP prevents feeding it the required patterns (Remark 3). The natural ranges in our range-respecting attacks were computed on test queries that include the attacked ones, and the adaptive game against value-aware samplers with held-out ranges is not yet run **[TODO: G3]**. We deliberately do not develop a method for steering the responses of instruction-tuned assistants.

9. Conclusion

A proof of inference that spot-checks an unencoded execution trace is not fully sound. Editing one value and recomputing the rest honestly turns the forged trace into the honest trace of a model that differs in one row, and finding that row is a search for a hidden object, in which no sampling rule, adaptive or value-aware, beats uniform sampling in the worst case. The object that must be encoded is therefore the model, not the trace. Our defence checks every weight operation of an exact integer model with Freivalds’ algorithm against a Reed–Solomon and Merkle commitment. It needs only a hash function and a linear code, stays sound against an untrusted committer with a one-time setup proof, binds the whole computation under Fiat–Shamir, and proves greedy and sampled generation. Its prover is far faster than published zkSNARK provers. Its cost is a proof that grows with the prompt, which V1 halves at 2,048 tokens but does not remove.

References

- [1] T. Liu, X. Xie, and Y. Zhang, “zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 2968–2985.
- [2] B.-J. Chen, S. Waiwitikhit, I. Stoica, and D. Kang, “ZKML: An optimizing system for ML inference in zero-knowledge proofs,” in *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM, 2024, pp. 560–574.
- [3] Zkonduit, “EZKL: An engine for doing inference for deep learning models and other computational graphs in a zk-SNARK,” <https://github.com/zkonduit/ezkl>, 2026, gitHub repository, accessed September 2026.
- [4] H. Sun, J. Li, and H. Zhang, “zkLLM: Zero knowledge proofs for large language models,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2024, pp. 4405–4419.
- [5] W. Qu, Y. Sun, X. Liu, T. Lu, Y. Guo, K. Chen, and J. Zhang, “zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference,” in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2045–2063.
- [6] N. Gailly, I. Hishon-Rezaizadeh, T. Liu, N. Mainardi, D. Papadopoulos, C. Papamanthou, C. Pappas, S. Srinivasan, Z. Youell, and Y. Zhang, “DeepProve: Verifiable end-to-end large language model inference,” Cryptology ePrint Archive, Paper 2026/1112, 2026, to appear in ACM CCS 2026.
- [7] B.-J. Chen, L. Tang, and D. Kang, “ZKTorch: Compiling ML inference to zero-knowledge proofs via parallel proof accumulation,” arXiv:2507.07031, 2025.
- [8] F. Tramèr and D. Boneh, “Slalom: Fast, verifiable and private execution of neural networks in trusted hardware,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [9] J. Xue, Y. Zhao, M. Zheng, F. Yao, Y. Solihin, and Q. Lou, “Securing transformer-based AI execution via unified TEEs and crypto-protected accelerators,” arXiv:2507.03278, 2025, presents TwinShield.
- [10] Z. Chen, M. Wu, M. Heinrich, J. Zeng, H. Lan, T. Zhang, and R. Tan, “Communication-efficient verifiable attention for LLM inference,” arXiv:2606.16352, 2026, presents VeriAttn.
- [11] P. Anchuri, M. Campanelli, P. Cesaretti, R. Gennaro, T. M. Jois, H. S. Kayman, and T. Ozdemir, “Towards verifiable AI with lightweight cryptographic proofs of inference,” in *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2026, pp. 955–977.
- [12] Y. Lim and S. Yong, “Seal, then sample: Sampled layerwise proofs for verifiable LLM inference from GPT-2 to 70B,” arXiv:2609.27367, 2026.
- [13] O. Baser, E. Sadeghi, E. Wang, N. Vergauwen, S. Kazemian, H. Kang, S. P. Chinchali, and S. Vishwanath, “TensorCommitments: A lightweight verifiable inference for language models,” arXiv:2602.12630v2, 2026.
- [14] Z. Wang, “NanoZK: Privacy-preserving verifiable inference for large language models via layerwise zero-knowledge proofs,” arXiv:2603.18046, 2026.
- [15] F. Carrone, D. Kingston, M. Puebla, and M. Toscano, “CommitLLM: A commit-and-audit protocol for open-weight LLM inference,” Preprint, <https://github.com/lambdaclass/CommitLLM>, 2026, paper at [paper/main.pdf](https://arxiv.org/abs/25541e83), commit 25541e83.
- [16] M. Dallachiesa, A. Pitasi, D. Pinger, J. Goodbody, and L. Vaello, “Statistical proof of execution (SPEX),” arXiv:2503.18899, 2025.
- [17] Y. Guo, W. Qu, L. Wu, S. Zhai, L. Z. Wang, M. Xu, Y. Liu, B. Yuan, D. Song, and J. Zhang, “IMMACULATE: A practical LLM auditing framework via verifiable computation,” arXiv:2602.22700, 2026.
- [18] A. Juels and J. Kaliski, Burton S., “PORs: Proofs of retrievability for large files,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2007.
- [19] F. Ergün, S. Kannan, S. R. Kumar, R. Rubinfeld, and M. Viswanathan, “Spot-checkers,” *Journal of Computer and System Sciences*, vol. 60, no. 3, pp. 717–751, 2000.
- [20] E. Ben-Sasson, O. Goldreich, P. Harsha, M. Sudan, and S. Vadhan, “Robust PCPs of proximity, shorter PCPs, and applications to coding,” *SIAM Journal on Computing*, vol. 36, no. 4, pp. 889–974, 2006.
- [21] R. Freivalds, “Probabilistic machines can use less running time,” in *Information Processing 77, Proceedings of the IFIP Congress*. North-Holland, 1977, pp. 839–842.
- [22] S. Ames, C. Hazay, Y. Ishai, and M. Venkatasubramanian, “Ligero: Lightweight sublinear arguments without a trusted setup,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 2087–2104.

- [23] D. Bernhard, O. Pereira, and B. Warinschi, “How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios,” in *Advances in Cryptology – ASIACRYPT 2012*. Springer, 2012, pp. 626–643.
- [24] Q. Dao, J. Miller, O. Wright, and P. Grubbs, “Weak Fiat-Shamir attacks on modern proof systems,” in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 199–216.
- [25] U. Haböck, “Multivariate lookups based on logarithmic derivatives,” Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [26] S. Papini and U. Haböck, “Improving logarithmic derivative lookups using GKR,” Cryptology ePrint Archive, Paper 2023/1284, 2023.
- [27] K. Gupta, J. Katz, and I. Miers, “Private and verifiable outsourcing of open-weight LLM inference,” Cryptology ePrint Archive, Paper 2026/1849, 2026.
- [28] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 38 087–38 099.
- [29] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, “Delegating computation: Interactive proofs for muggles,” in *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 2008, pp. 113–122.
- [30] T. Xie, T. Lu, Z. Fang, S. Wang, Z. Zhang, Y. Jia, D. Song, and J. Zhang, “zkPyTorch: A hierarchical optimized compiler for zero-knowledge machine learning,” Cryptology ePrint Archive, Paper 2025/535, 2025.
- [31] W. Qu, Y. Guo, Y. Ying, and J. Zhang, “VerfCNN: Optimal complexity zkSNARK for convolutional neural networks,” Cryptology ePrint Archive, Paper 2025/2020; IEEE Symposium on Security and Privacy (S&P) 2026, 2025.
- [32] S. Lee, H. Ko, J. Kim, and H. Oh, “vCNN: Verifiable convolutional neural network based on zk-SNARKs,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.
- [33] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu, “ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences,” Cryptology ePrint Archive, Paper 2021/087, 2021. [Online]. Available: <https://eprint.iacr.org/2021/087>
- [34] B. Feng, Z. Wang, Y. Wang, S. Yang, and Y. Ding, “ZENO: A type-based optimization framework for zero knowledge neural network inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2024, pp. 450–464.
- [35] D. Zakharov, O. Kurbatov, A. Sdobnov, L. Soukhanov, Y. Sekhin, V. Volovyk, M. Velykodnyi, M. Cherepovskyi, K. Baibula, L. Antadze, P. Kravchenko, V. Dubinin, and Y. Panasenko, “Bionetta: Efficient client-side zero-knowledge machine learning proving,” arXiv:2510.06784, 2025, arXiv version 2 (8 Dec 2025), whose Table 4 gives the numbers we use.
- [36] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, “Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning,” in *30th USENIX Security Symposium*. USENIX Association, 2021, pp. 501–518.
- [37] W. Benno, A. Centelles, A. Douchet, and K. Gibran, “Jolt atlas: Verifiable inference via lookup arguments in zero knowledge,” arXiv:2602.17452, 2026.
- [38] L. Wang, H. Lou, C. Li, Y. Yu, and Y. Hu, “zkAgent: Verifiable LLM agent execution via one-shot transcript proofs,” Cryptology ePrint Archive, Paper 2026/199, 2026.
- [39] C. Gong, B. Liu, and M. Li, “Hollow-LLM attack: Computationally trivial weights in zero-knowledge verification of LLM inference,” arXiv:2607.28884; IEEE Symposium on Security and Privacy (S&P) 2026, 2026.
- [40] J. M. Ong, M. Di Ferrante, A. Pazdera, R. Garner, S. Jaghouar, M. Basra, M. Ryabinin, and J. Hagemann, “TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025, pp. 47 196–47 211.
- [41] Y. Sun, Y. Li, Y. Zhang, Y. Jin, and H. Zhang, “SVIP: Towards verifiable inference of open-source large language models,” arXiv:2410.22307, 2024.
- [42] A. Arun, A. St. Arnaud, A. Titov, B. Wilcox, V. Kolobaric, M. Brinkmann, O. Ersoy, B. Fielding, and J. Bonneau, “Verde: Verification via refereed delegation for machine learning programs,” arXiv:2502.19405, 2025.
- [43] K. Conway, C. So, X. Yu, and K. Wong, “opML: Optimistic machine learning on blockchain,” arXiv:2401.17555, 2024.
- [44] J. Yao, H. Su, T. Liao, Z. Cheng, H. Zhang, X. Wang, and P. Viswanath, “TAO: Tolerance-aware optimistic verification for floating-point neural networks,” in *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. ACM, 2026, pp. 1515–1532.
- [45] G. Ateniese, R. Burns, R. Curtmola, J. Herring, L. Kissner, Z. Peterson, and D. Song, “Provable data possession at untrusted stores,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2007.
- [46] L. Babai, L. Fortnow, L. A. Levin, and M. Szegedy, “Checking computations in polylogarithmic time,” in *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1991, pp. 21–31.
- [47] A. Chiesa, D. Ojha, and N. Spooner, “Fractal: Post-quantum and transparent recursive proofs from holography,” in *Advances in Cryptology – EUROCRYPT 2020*. Springer, 2020.
- [48] J. B. Kadane, “Optimal whereabouts search,” *Operations Research*, vol. 19, no. 4, pp. 894–904, 1971.
- [49] H. Jia, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran, and N. Papernot, “Proof-of-learning: Definitions and practice,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021.
- [50] R. Zhang, J. Liu, Y. Ding, Z. Wang, Q. Wu, and K. Ren, “adversarial examples” for proof-of-learning,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022.
- [51] C. Fang, H. Jia, A. Thudi, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, V. Chandrasekaran, and N. Papernot, “Proof-of-learning is currently more broken than you think,” in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023.
- [52] Z. Ghodsi, T. Gu, and S. Garg, “SafetyNets: Verifiable execution of deep neural networks on an untrusted cloud,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [53] B. Merbaum, M. A. Raeisi, W. Wang, C. Papamanthou, K. Sotiraki, and F. Zhang, “Maverick: Private and verifiable LLM inference made practical via matrix-vector multiplication delegation,” arXiv:2609.10264, 2026.
- [54] K. Lee, B. Han, J. Kim, and H. Oh, “LAMP: Linear verification of matrix multiplication via proximity testing,” Cryptology ePrint Archive, Paper 2026/1571, 2026.
- [55] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, “Brakedown: Linear-time and field-agnostic SNARKs for RICS,” in *Advances in Cryptology – CRYPTO 2023*. Springer, 2023, pp. 193–226.
- [56] H. Zeilberger, B. Chen, and B. Fisch, “BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes,” in *Advances in Cryptology – CRYPTO 2024*, ser. LNCS, vol. 14929. Springer, 2024, pp. 138–169.
- [57] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology – CRYPTO ’87*, ser. LNCS, vol. 293. Springer, 1987, pp. 369–378.

- [58] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, “BadNets: Evaluating backdoor attacks on deep neural networks,” *IEEE Access*, vol. 7, pp. 47 230–47 244, 2019.
- [59] J. Chew, Milton C., “A sequential search procedure,” *The Annals of Mathematical Statistics*, vol. 38, no. 2, pp. 494–502, 1967.
- [60] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [61] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.
- [62] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Federal Information Processing Standards Publication 202, 2015.
- [63] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology – CRYPTO ’86*, ser. LNCS, vol. 263. Springer, 1986, pp. 186–194.
- [64] J.-S. Coron, Y. Dodis, C. Malinaud, and P. Puniya, “Merkle–Damgård revisited: How to construct a hash function,” in *Advances in Cryptology – CRYPTO 2005*, ser. LNCS, vol. 3621. Springer, 2005, pp. 430–448.
- [65] R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, R. D. Rothblum, and D. Wichs, “Fiat-Shamir: From practice to theory,” in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*. ACM, 2019, pp. 1082–1090.
- [66] J. Holmgren, “On round-by-round soundness and state restoration attacks,” *Cryptology ePrint Archive*, Paper 2019/1261, 2019.
- [67] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *Theory of Cryptography (TCC 2016-B)*, ser. LNCS, vol. 9986. Springer, 2016, pp. 31–60.
- [68] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, “Proximity gaps for Reed–Solomon codes,” in *IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2020, pp. 900–909.
- [69] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, “Algebraic methods for interactive proof systems,” *Journal of the ACM*, vol. 39, no. 4, pp. 859–868, 1992.
- [70] P. Tillet, H. T. Kung, and D. Cox, “Triton: An intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. ACM, 2019, pp. 10–19.
- [71] P. L. Montgomery, “Modular multiplication without trial division,” *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [72] E. J. Gumbel, *Statistical Theory of Extreme Values and Some Practical Applications: A Series of Lectures*, ser. National Bureau of Standards Applied Mathematics Series. Washington, DC: U.S. Government Printing Office, 1954, no. 33.
- [73] C. J. Maddison, D. Tarlow, and T. Minka, “A* sampling,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.
- [74] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, “OPT: Open pre-trained transformer language models,” arXiv:2205.01068, 2022.
- [75] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [76] Z. Yao, X. Wu, C. Li, S. Youn, and Y. He, “ZeroQuant-V2: Exploring post-training quantization in LLMs from comprehensive study to low rank compensation,” arXiv:2303.08302, 2023.
- [77] S. Son, W. Park, W. Han, K. Kim, and J. Lee, “Prefixing attention sinks can mitigate activation outliers for large language model quantization,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [78] Y. Dodis, R. Ostrovsky, L. Reyzin, and A. Smith, “Fuzzy extractors: How to generate strong keys from biometrics and other noisy data,” *SIAM Journal on Computing*, vol. 38, no. 1, pp. 97–139, 2008.

Appendix A.

Spot-Checking: Full Statements and Proofs

Setting. Fix a target layer with rows $\theta = (\theta_1, \dots, \theta_N)$, all sets finite, as with int8 or fp16 values. Site i has value $h_i = f_i(\theta_i)$, for example $f_i(\theta_i) = \phi(w_i \cdot a + b_i)$ with a the layer’s input on x . The honest trace is $\tau(\theta)$. The view is $P = (x, Z, \tau')$, and $\mathcal{A}$ is the set of admissible instances, those consistent with Z . A verifier $V \in \text{Spot}_d(q)$ makes queries to a commitment oracle $\Sigma(\theta)$ of locality d , chosen adaptively. It accepts $(\tau(\theta), \Sigma(\theta))$ for every $\theta \in \mathcal{A}$ and every coin string (completeness), and the expected number of its queries that meet the target layer is at most q (under each reference oracle R_p below). The adversary is a randomised rule A that sees θ and outputs a pair (u, v) with $v \neq h_u$, or $\perp$. Its forgery is $\tau' = \tau$ with h_u replaced by v and every later layer recomputed with the true weights. We write J for the random edited site, and we condition throughout on $A \neq \perp$.

Definition 1 (explainable forgery). A forgery (u, v) on θ is *explainable* if there is $\theta' \in \mathcal{A}$ with $\theta'_{-u} = \theta_{-u}$ and $\tau(\theta') = \tau'$.

For $f_u = \text{ReLU}(w_u \cdot a + b_u)$, the row $\theta'_u = (\mathbf{0}, v)$ for $v > 0$, or $(\mathbf{0}, -1)$ for $v = 0$, explains every edit. If V range-checks the trace against published intervals and $\mathcal{A}$ is the set of instances whose honest trace lies in range, then every edit whose forged values all lie in range is explainable by the same row. If Z fixes each neuron’s full natural distribution, explainability can fail, and Proposition 1 then constructs an instance where it holds. For SiLU or GELU, v must lie in the image of the activation, up to rounding. If the trace stores pre-activations, edit z_u instead.

Lemma 2 (no touch, no rejection). *In the ideal-oracle model, if a forgery at u is explainable, then every $V \in \text{Spot}_d(q)$ that does not touch u accepts it.*

Proof. Fix the coins. While no query touches u , the answers $\gamma_\sigma(\theta_{S_\sigma})$ and $\gamma_\sigma(\theta'_{S_\sigma})$ coincide. By induction on the queries, the run on $(P, \Sigma(\theta))$ therefore equals the run on $(P, \Sigma(\theta'))$. The latter is an honest run of $\theta' \in \mathcal{A}$, which V accepts by completeness. $\square$

Definition 2 (location-oblivious rows, (H)). The pair (prior, A) satisfies (H) if for every view p there is an oracle R_p , which may be sampled lazily, with the following property. For every u with $\Pr[J = u \mid P = p] > 0$ and every verifier, the run of V under $\Sigma(\theta) \mid (P = p, J = u)$, up to and including the *choice* of its first query that touches u (but not that query’s answer), has the same law as under R_p .

For $t \geq 0$ let $G_t(J | P) = \mathbb{E}_p \max\{\sum_u x_u \Pr[J = u | P = p] : x \in [0, 1]^N, \sum_u x_u \leq t\}$. For integer t this is the expected total of the t largest posteriors.

Theorem 6 (full form of Theorem 1). *Assume (H) and that every forgery in the support of A is explainable. In the ideal-oracle model, every $V \in \text{Spot}_d(q)$ satisfies*

$$\Pr[V \text{ rejects}] \leq G_{qd}(J | P) \leq qd \cdot 2^{-\tilde{H}_\infty(J|P)},$$

where $\tilde{H}_\infty$ is average min-entropy [78]. Under the second commitment condition of Remark 1, add $2Q2^{-\kappa} + Q^22^{-\ell_H}$. For $d = 1$ and integer q , a verifier that knows the prior and A , and has a binding commitment, attains the bound with equality.

Proof. By Lemma 2, $\Pr[\text{reject}] \leq \sum_p \Pr[p] \sum_u \Pr[u | p] \Pr[V \text{ touches } u | p, u]$. Whether V touches u is determined by its run up to the choice of the first query that touches u . By (H), $\Pr[V \text{ touches } u | p, u] = \Pr_{R_p}[V \text{ touches } u] =: x_u$, and x does not depend on u . Under R_p , V touches at most d sites per query that meets the layer, so $\sum_u x_u \leq qd$ and $x \in [0, 1]^N$. Hence $\sum_u \Pr[u | p] x_u$ is at most the value of the fractional knapsack $\max\{\sum_u x_u \Pr[u | p] : \sum_u x_u \leq qd\}$, which averages to $G_{qd}(J | P)$. The second inequality bounds each of the qd terms by the largest posterior, and $\mathbb{E}_p \max_u \Pr[u | p] = 2^{-\tilde{H}_\infty(J|P)}$. For tightness, the verifier that opens the q sites with the largest posteriors accepts honest traces. On a forgery it rejects exactly when J is among them, because binding makes the opened row θ_J show $f_J(\theta_J) = h_J \neq v$, while every other opened row is consistent. This is the optimal policy for finding a hidden object within q looks, given that a clean opened row is an empty box [48], [59]. $\square$

Lemma 3 (independent rows give (H)). *Suppose the prior on θ given Z is a product $\bigotimes_j \mu_j$, and A depends on the target rows only through their values: the law of $A(\theta)$ is a function of $h(\theta)$. Then (H) holds with $R_p = \bigotimes_j \mu_j(\cdot | f_j(\theta_j) = p_j)$.*

Proof. Write $\alpha(h, u, v) = \Pr[A(h) = (u, v)]$. On the event $\{P = p, J = u\}$ we have $h_j = p_j$ for all $j \neq u$, so $\Pr[\theta, J = u, P = p] = \prod_{j \neq u} \mu_j(\theta_j) \mathbf{1}[f_j(\theta_j) = p_j] \cdot \mu_u(\theta_u) \alpha((f_u(\theta_u), p_{-u}), u, p_u)$. This is a function of θ_{-u} times a function of θ_u . Hence, given $(P = p, J = u)$, $\theta_{-u} \sim \bigotimes_{j \neq u} \mu_j(\cdot | f_j = p_j)$, which is R_p restricted to the rows other than u . A run that has not yet touched u reads only θ_{-u} . $\square$

Lemma 3 concerns a *random ensemble* of networks. Averaging over it shows that every architecture has a bad network. It says nothing about a particular trained network, whose rows are correlated. The single-site attack satisfies the hypothesis on A : it ranks neurons by the margin gradient with respect to h , which uses only the later weights, and bisects by recomputing the later layers.

Corollary 1 (the posterior on the edited site). *Under Lemma 3, let π_u be the law of $f_u(\theta_u)$ when $\theta_u \sim \mu_u$, and let $\beta_u(p) = \Pr_{h_u \sim \pi_u}[A(h_u, p_{-u}) = (u, p_u)]$. Then*

$\Pr[J = u | P = p] \propto \beta_u(p)/\pi_u(p_u)$. *If $\pi_u(p_u) = 0$, the posterior is a point mass on u .*

Proof. Summing the factorisation above over θ gives $\Pr[J = u, P = p] = (\prod_j \pi_j(p_j)) \beta_u(p)/\pi_u(p_u)$. If $\pi_u(p_u) = 0$, then $\Pr[J = u', P = p] = 0$ for every $u' \neq u$, since that would require $h_u = p_u$. $\square$

The formula has two factors. Writing a common value makes $1/\pi_u$ small, but it hides the site only if the *choice* of site, β , is also spread out. A published, deterministic attack rule can concentrate β on one site even when every value it writes is common. Moreover, π_u is a law over *rows* at a fixed query, not the distribution of activations over *inputs* that our measurements report (65% zeros). The corollary is therefore a design rule, and the measured numbers are consistent with it rather than predicted by it. In exact computations on 12 random independent-row instances ($N = 4$, $q \in \{1, 2\}$), the posterior matched Corollary 1 and the optimal adaptive verifier matched G_q in every case.

Proposition 1 (the hard instance; Theorem 2(a) in full). *Let $N \geq 2$, b bits per weight, $c \geq \kappa/b$ dead input coordinates (equal to 0 on every natural input), and σ a uniform permutation of $[N]$. Hidden neuron i has the row $e_{\sigma(i)} + \sum_{l \leq c} r_{i,l} e_{\text{dead},l}$ with bias 0, where the $r_{i,l}$ are uniform and independent. Let $g = \text{ReLU}(\sum_i h_i - (N - 1))$ and let the logits be $(1/2, g)$. Natural inputs are uniform over $\mathbf{1}$ and the N vectors with one zero, with every dead coordinate 0. Then for every $V \in \text{Spot}_d(q)$ that knows the later weights, all natural ranges and the natural distribution of every hidden neuron, $\mathbb{E}_{\sigma, r} \Pr[V \text{ accepts}] \geq 1 - qd/N$ in the ideal-oracle model, and $\geq 1 - qd/N - 2Q2^{-\kappa} - Q^22^{-\ell_H}$ for an unsalted Merkle tree with ℓ_H -bit hashes and Q hash evaluations by V . In particular some (σ, r) achieves the bound, and a single random draw defeats every verifier simultaneously.*

Proof. On the input $\mathbf{1}$ we have $g = 1$. On a query whose zero is at ζ , $g = 0$, and setting $h_J = 1$ with $J = \sigma^{-1}(\zeta)$ gives $g = 1$, which flips the class. Every forged value lies in $\{0, 1\}$, the natural range. The forged trace, the later weights, the ranges and each hidden neuron's marginal (zero on exactly one of the $N+1$ natural inputs) do not depend on (σ, r) . Replacing row J by $e_j + \sum_l r'_l e_{\text{dead},l}$, for some $j \neq \zeta$ and fresh r' , gives an admissible instance whose honest trace is τ' , so the forgery is explainable. For (H), let R_p give each newly touched row a uniform unused element of $[N] \setminus \{\zeta\}$ and fresh r . Given $J = u$, the other rows form a uniform bijection onto $[N] \setminus \{\zeta\}$ with independent r , so revealing them one at a time has the law of R_p , as long as fewer than N rows have been touched, which holds whenever $qd < N$ (otherwise the claim is trivial). J is uniform given the view, so Theorem 6 gives detection at most qd/N . For the Merkle tree, apply Remark 1: each leaf preimage, under the true rows and under the explanation, has min-entropy at least κ given the view and the other leaves. $\square$

Proposition 2 (value-blind samplers on every fixed network). *Let V reject only on a failed local check, and*

let its choice of rows ignore the claimed values (it may depend on the rows already opened). On a fixed network, let $B_1, \dots, B_m$ be disjoint sets of sites with forgeries τ'_j whose inconsistent sites are exactly B_j . Then $\min_j \Pr[V \text{ rejects } \tau'_j] \leq q/m$. If the choice is deterministic, every forgery whose set B_j avoids the chosen rows is accepted with probability 1.

Proof. On a fixed network the opened rows are the true ones whatever the forgery, and the choice ignores the claimed values. So, up to the first opening inside B_j , the run on τ'_j equals one reference run whose opened set I has $\mathbb{E}|I| \leq q$. Then $\Pr[\text{reject } \tau'_j] \leq \Pr[I \cap B_j \neq \emptyset] \leq \mathbb{E}|I \cap B_j|$, and these sum over j to at most q , by disjointness. $\square$

Proposition 3 (mixing suffices). *Let a weight layer compute $Z = A[X; \mathbf{1}]$ with $A \in \mathbb{F}_p^{N \times k}$, where the layer's integer arithmetic embeds in $\mathbb{F}_p$ without wrap-around (the range checks of Section 4). Let each row of A be Reed–Solomon encoded to length n and committed column by column, and assume the committed matrix is the encoding of A , either because C_M is computed honestly or because the setup proof of Section 4.5 has been checked (then replace $k-1$ by $k-1+e$ below, where e is that proof's radius). The verifier draws r vectors $\chi \in \mathbb{F}_p^N$ after the claims Z are fixed, receives a folded row u for each (honestly, $u = \chi^\top A$), checks $\chi^\top Z = u[X; \mathbf{1}]$, and then opens t distinct uniform columns c , checking $\chi^\top \hat{A}_c = \text{Enc}(u)_c$; under Fiat–Shamir the challenges are derived in the same order. Each folded row and each column symbol has support $[N]$. If $Z \neq A[X; \mathbf{1}]$, the layer passes with probability at most $p^{-r} + ((k-1)/n)^t$, plus the probability of a hash collision. With $r = \lceil \beta / \log_2 p \rceil$ and $t = \lceil \beta / \log_2(n/(k-1)) \rceil$ this is at most $2^{1-\beta}$. For any row-wise linear code of relative distance δ , the second term is $(1-\delta)^t$. The verifier's work per layer is $O(r(N+k)T + rtN + rn \log n)$, against $O(NkT)$ for recomputation, and it touches every row.*

Proof. Let $D = Z - A[X; \mathbf{1}] \neq 0$, with $D_{ij} \neq 0$. If every received u equals $\chi^\top A$, the two sides of each check differ by $\chi^\top D$, whose entry j is $\chi_i D_{ij} + c$ with c independent of χ_i ; it vanishes for one value of χ_i only, so all r checks pass with probability at most p^{-r} . If some $u \neq \chi^\top A$, then $\text{Enc}(u)$ and $\chi^\top \hat{A} = \text{Enc}(\chi^\top A)$ are distinct codewords, which agree on at most $k-1$ of the n positions; this set is fixed before the columns are drawn, and t distinct uniform columns all fall in it with probability at most $((k-1)/n)^t$. Unless the prover finds a hash collision, the opened columns are the committed ones. The work bound counts the two sides of each check, the t opened columns of N entries for each fold, and one encoding per fold. $\square$

Proof of Theorem 2. (a) Proposition 1 gives acceptance at least $1 - qd/N$ for a commitment that hides unopened rows. For an unsalted Merkle tree it gives $1 - qd/N - 2Q2^{-\kappa} - Q2^{2-\ell_H}$, where the term in κ can be made as small as desired. If the soundness error is at most ε on every such network, then $1 - qd/N \leq \varepsilon$, that is, $qd \geq (1-\varepsilon)N$. A path visits one node of the target layer and opens that node's

row, so s paths make s queries there with $d = 1$. (b) Set $\beta = \lambda + \log_2(2L)$ in Proposition 3. Each layer then passes a wrong claim with probability at most $2^{1-\beta} = 2^{-\lambda}/L$, and a union bound over the L layers, as in the soundness theorem of Section 4, gives $2^{-\lambda}$. Each layer uses $r + t = O(\beta) = O(\lambda + \log L)$ queries of support N . $\square$

Part (a) is the one-sided randomised query complexity of OR_N with d -row queries, placed inside a network; equivalently, an input oracle at relative distance $1/N$ from the language cannot be rejected with fewer than $(1-\varepsilon)N/d$ queries [20].

Remark 1 (commitments). The lower bounds need the commitment to *hide* unopened rows. Binding is used only for tightness and for Proposition 3. A Merkle opening includes an authentication path whose sibling hashes depend on other rows, so with unsalted leaves and low-entropy rows a verifier can brute-force sibling subtrees. On the instance of Proposition 1 without the random weights, opening one row and inverting the hashes of the sibling subtrees up to height η detects the edit with probability $2^{\eta+1}/N$ instead of $1/N$ (exact computation for $N = 16$: 0.125, 0.25 and 0.5 for $\eta = 0, 1, 2$, against 0.0625). Lemma 2 and Theorem 6 therefore hold in either of two cases. The first is a hiding vector commitment, for example Merkle leaves $H(s_i \| \theta_i)$ with fresh salts. The second is the random-oracle model with an unsalted tree, provided every leaf preimage (under the true rows and under the explanation) has min-entropy at least κ given the view and the other leaves. In the second case add $2Q2^{-\kappa} + Q2^{2-\ell_H}$: the coupling of the two runs fails only if a hash query hits one of the two preimages of leaf u , or if two hash queries collide. Trained weights have high-entropy rows, so this concerns the proof, not the attack or the measurements.

Remark 2 (when adaptivity helps: correlated rows). Take $m+1$ sites. Row 0 stores the index z of the critical site in weights on dead inputs, so its value $h_0 = 0$ reveals nothing. Sites $1, \dots, m$ carry bits with the only zero at z , and the output is their AND. The forged trace (all ones) is the same for every z , so $G_2 = 2/m$. An adaptive verifier opens row 0, reads z , opens row z and catches the edit with certainty. Correlated rows thus act as a crude encoding, and (H) cannot be dropped from Theorem 6. This example is analytic. In exact toy computations, simple copy correlation ($\theta_1 = \theta_0$) produced no gap in 12 instances.

Remark 3 (scope). (i) *Imperfect completeness* c adds $1-c$. (ii) *Multi-site edits*: if the inconsistent set B is explainable by changing the rows of B alone, replace “touch J ” by “touch B ”. Proposition 2 is the value-blind case. (iii) *Sites need not be neurons*: any partition into units with unit-local evidence fits, such as proof chunks [12] or (layer, token) cells [15]. (iv) *Repeated queries*: the bound is per query and assumes a *stateless* verifier. A verifier that sees whole traces and caches opened rows checks them for free on later queries, and after about N/q queries it has in effect downloaded the layer, which amounts to holding the weights. The path protocol opens trace values only along its

TABLE 10. PATHS THE PATH PROTOCOL NEEDS TO CATCH k EDITED NEURONS OF ONE FFN LAYER WITH PROBABILITY $1 - 2^{-40}$ (LOWER BOUNDS, FROM $(1 - k/d_{\text{ff}})^s \leq 2^{-40}$)

k	Qwen3-4B-Instruct-2507 ($d_{\text{ff}} = 9,728$)	Qwen2.5-7B-Instruct ($d_{\text{ff}} = 18,944$)
1	269,704	525,226
2	134,845	262,606
4	67,416	131,296
8	33,701	65,642
16	16,844	32,814

path, so caching rows does not let it check unvisited nodes, and $1 - (1 - 1/N)^k$ stays the bound for k chosen tampered queries. (v) *Transformers*: padding embeds the instance of Proposition 1 in every ReLU MLP with input dimension and a hidden width of at least N and one further hidden layer. In a transformer, the LayerNorm before the MLP prevents feeding the 0/1 patterns directly. We do not claim the embedding there. (vi) *Side information*: Proposition 1 allows the later weights, the ranges and the target layer’s marginals. It does not cover joint statistics, or the full marginals of later neurons.

Instruction-tuned models. Under the path protocol, detection depends only on how many sites are edited, not on what the edit does. A path visits at most one neuron of a given FFN layer, and when it enters that layer it picks the neuron uniformly among the d_{ff} neurons. So k edited neurons in that layer are detected with probability at most k/d_{ff} per path, and the edit passes s paths with probability at least $(1 - k/d_{\text{ff}})^s$. Table 10 gives the resulting number of paths needed for detection probability $1 - 2^{-40}$ on two instruction-tuned models. It follows from the formula and the models’ shapes, and needs no attack search. We measured $k = 1$ for next-token changes on OPT-6.7B (Section 3.5). We deliberately do not develop or release a method for steering whole responses of aligned assistants: the integrity failure is already characterised by the acceptance bound, and such a method would mainly help a dishonest provider.

Appendix B.

The Defence: Integer Semantics and Proofs

B.1. Integer Semantics

All values are integers, and every multiplier, gain and table below is a public constant of the architecture, bound by the graph digest of Section 4.4.

Requantisation and residuals. With a public multiplier m , a requantisation computes $\text{clamp}(\lfloor (mz + 2^{29})/2^{30} \rfloor, -127, 127)$, which rounds to the nearest integer with ties upwards. A residual addition computes $\text{clamp}(a + \lfloor (mz + 2^{29})/2^{30} \rfloor, 1 - 2^{22}, 2^{22} - 1)$, so the residual stream holds 23-bit integers.

Norms. Over the d channels of one position, LayerNorm first subtracts the integer mean $\lfloor (\sum_i x_i + \lfloor d/2 \rfloor)/d \rfloor$; RMSNorm does not. Both compute $\sigma =$

$\max(1, \lfloor \sqrt{(\sum_i x_i^2)/d} \rfloor)$, evaluated in floating point and corrected by one integer step in each direction, which makes it exact, and output $\text{clamp}(\lfloor (g_i x_i + 2^{15}\sigma)/(2^{16}\sigma) \rfloor, -127, 127)$ for a public integer gain g_i per channel.

Activations. ReLU, GELU and SiLU look up an int8 value, clamped to $[-128, 127]$, in a public table of 256 entries. SwiGLU multiplies the SiLU output by the up projection and requantises the product.

Rotary embeddings. At position t , with angles $t\theta_j$ that follow from the public base, let $C = \text{round}(2^{14} \cos t\theta_j)$ and $S = \text{round}(2^{14} \sin t\theta_j)$. A pair (x_1, x_2) becomes $\lfloor (x_1 C - x_2 S + 2^{13})/2^{14} \rfloor$ and $\lfloor (x_2 C + x_1 S + 2^{13})/2^{14} \rfloor$, each clamped to $[-127, 127]$.

Attention. The scores are $s = qk^\top$ for int8 q and k . In each query row the verifier forms the gaps $g_j = \max_{j'} s_{j'} - s_j$ over the keys that the causal mask allows, the indices $i_j = \text{clamp}(\lfloor (m_s g_j + 2^{29})/2^{30} \rfloor, 0, 255)$ for a public m_s , and $e_j = E[i_j]$ with $E[i] = \text{round}(2^{15} e^{-i/16})$; masked keys get $e_j = 0$. With $S = \sum_j e_j \geq 2^{15}$, the probabilities are $P_j = \lfloor (510e_j + S)/(2S) \rfloor \in [0, 255]$, that is, $255e_j/S$ rounded, and the output PV is requantised. Grouped-query attention shares each key and value head among its query heads.

Exact hardware arithmetic. Weight products run on int8 tensor cores with 32-bit accumulation over blocks of 2^{16} products, each at most 2^{14} in magnitude, so every block sum is at most 2^{30} and exact in any order; blocks are added in 64-bit integers. The attention products run in float32, which is exact on integers up to 2^{24} . With head dimension at most 128, every score is at most $128 \cdot 128^2 = 2^{21}$ in magnitude. A row of P sums to at most $255 + T/2$, so every partial sum of PV is below $(255 + T/2) \cdot 128 < 2^{24}$ for $T < 2^{15}$; longer rows use 64-bit integers. Before using its int8 units, the prover multiplies random and extreme operands (every entry -128 , and sums at the 2^{30} bound) and compares the results with an exact reference; on a mismatch it falls back to an exact float64 path.

B.2. Proof of Theorem 3

Fix x and the prover’s first message. Let $\tilde{X}_\ell$ be the input of operation ℓ that the verifier derives from x and the claims, and $D_\ell = Z_\ell - A_\ell[\tilde{X}_\ell; \mathbf{1}]$ over $\mathbb{F}_p$.

The first wrong operation. If every claim equals the value of M , then $y = M(x)$. Otherwise let ℓ be the first operation, in graph order, whose claim differs from the value of M . All earlier claims are correct and $\tilde{X}_\ell$ is computed from them by public operations, so $\tilde{X}_\ell$ is the true input. The wrong claim passed the range check, $|z| < 2^{29}$, and the true value meets the bound checked at setup, so the two integers differ by less than $2^{30} < p$ and $D_\ell \neq 0$. A lookup cannot be the first wrong operation, since a different row would need a collision; so ℓ belongs to a matrix $j \in \mathcal{J}$, which the first message determines.

Row layout. If $u_\ell = \chi_\ell^\top A_\ell$, the check of Eq. (2) passes only if $\chi_\ell^\top D_\ell = 0$, which has probability at most p^{-r} , as shown in Section 4.3; χ_ℓ is drawn after the claims. Otherwise some row ρ of u_ℓ differs from row ρ of $\chi_\ell^\top A_\ell$.

Unless the prover finds a collision, every opened column that passes the Merkle check is the honest column $\hat{A}_{\ell,c}$, and the check at c compares $\text{Enc}(u_{\ell,\rho})_c$ with $\text{Enc}((\chi_\ell^\top \hat{A}_\ell)_\rho)_c$. These are distinct codewords, and two distinct polynomials of degree below k_j agree on at most $k_j - 1$ of the n_j points. The t_j distinct uniform columns are drawn after u_ℓ , so they all fall into this set with probability at most $\prod_{i < t_j} (k_j - 1 - i)/(n_j - i)$. In a shared tree each member's check uses its own projection of the opened leaves, and the same count applies.

Transposed layout. Let j stack the operations that read $\tilde{X} = \tilde{X}_\ell$, with stacked weights A (k_j rows), claims Z and error $D \neq 0$. The committed matrix $\hat{A}'$ holds the encodings of the columns of A . For each column χ'_i of χ' the verifier computes $z_i = Z\chi'_i$ and $w_i = [\tilde{X}; 1]\chi'_i$ and checks $\text{Enc}(z_i)_c = w_i^\top \hat{A}'_c$ on the opened columns. Since $w_i^\top \hat{A}' = \text{Enc}(Aw_i)$ and $Aw_i = A[\tilde{X}; 1]\chi'_i$, correct claims pass. If $D\chi'_i = 0$ for every i , then χ' is not the identity, and a nonzero row d of D satisfies $d^\top \chi'_i = 0$ for r independent uniform vectors, which has probability at most p^{-r} . Otherwise $\text{Enc}(z_i)$ and $\text{Enc}(Aw_i)$ are distinct codewords of message length k_j for some i , and the column term applies as above.

Modes K and K_{pre} . In K the verifier computes u_ℓ with a fresh χ_ℓ , so only the first case arises, with probability at most p^{-r} . In K_{pre} the vectors are fixed and secret. Call a check of operation ℓ in some query *bad* if $D_\ell \neq 0$ and the check passes. Without a bad check, a check passes exactly when $D_\ell = 0$, so every outcome, and whichever check the verifier reports as failed, is a function of the prover's messages. Consider a hybrid verifier that reports these predicted outcomes and never reads the vectors. Against it, the prover's messages are independent of the vectors, so a given check with $D_\ell \neq 0$ is bad with probability at most p^{-r} , and some check among the at most Q queries is bad with probability at most $Q \sum_\ell p^{-r}$. The real and hybrid runs agree until the first bad check, and an accepted $y \neq M(x)$ requires a bad check of its first wrong operation.

Round-by-round soundness. Call a partial transcript *doomed* if (i) after the claims, $y \neq M(x)$; (ii) after the vectors, some row-layout matrix has $\chi_j^\top D_j \neq 0$ or some transposed matrix has $D_j \chi'_j \neq 0$; (iii) after the folded rows, Freivalds' check fails for some operation, or some row-layout matrix has $u_j \neq \chi_j^\top A_j$, or some transposed matrix has $D_j \chi'_j \neq 0$. From (i), the vectors avoid (ii) with probability at most p^{-r} (consider the matrix of the first wrong operation). From (ii), every folded row the prover can send leads to (iii). From (iii), either a check fails or some matrix j escapes only if all its columns fall into an agreement set, which has probability at most its column term. A complete doomed transcript is rejected unless the prover finds a collision. The round-by-round error is therefore at most $\max(p^{-r}, \max_j \prod_{i < t_j} (k_j - 1 - i)/(n_j - i)) \leq \varepsilon_C$, and the Fiat-Shamir bound of Section 4.4 follows from [65], [67]. $\square$

B.3. Proof of Fact 1

The map $(\chi_1, \dots, \chi_s) \mapsto \sum_i \xi^{i-1} \chi_i$ is a bijection from $(\mathbb{F}_p^R)^s$ to $\mathbb{K}_s^R$, so ψ is uniform. Every $v \in \mathbb{K}_s^n$ is uniquely $\sum_i \xi^{i-1} v_i$ with $v_i \in \mathbb{F}_p^n$. The evaluation points lie in $\mathbb{F}_p$, so a polynomial $P = \sum_i \xi^{i-1} P_i$ over $\mathbb{K}_s$ of degree below k , with each P_i over $\mathbb{F}_p$ of degree below k , has evaluations $\sum_i \xi^{i-1} P_i(\cdot)$; the code over $\mathbb{K}_s$ is thus $\{\sum_i \xi^{i-1} c_i\}$ with every c_i in the code over $\mathbb{F}_p$. Since $\psi^\top E = \sum_i \xi^{i-1} \chi_i^\top E$, it agrees with $\sum_i \xi^{i-1} c_i$ at a column exactly when every $\chi_i^\top E$ agrees with c_i there. $\square$

B.4. Proof of Theorem 4

Parameters. Let $\beta_0 = \lambda + 64 + \log_2(2|\mathcal{J}|)$ and $s = \lceil (\beta_0 + \log_2 \max_j n_j) / \log_2 p \rceil$, so that $n_j/p^s \leq 2^{-\beta_0}$. Each tree takes the radius $e_j = \lfloor f(n_j - k_j)/2 \rfloor$ for all its members, with $f \in \{2^0, \dots, 2^{-10}\}$ chosen to minimise $t^0/a + t$; t^0 is the least integer with $\prod_{i < t^0} (n_j - e_j - 1 - i)/(n_j - i) \leq 2^{-\beta_0}$, and the per-query t the least with $\prod_{i < t} (k_j + e_j - 1 - i)/(n_j - i) \leq 2^{-\beta}$, both for every member. Then $\varepsilon_0 \leq Q \cdot 2|\mathcal{J}| \cdot 2^{-\beta_0} + O(Q^2 2^{-256}) \leq 2^{-\lambda} + O(Q^2 2^{-256})$ for $Q \leq 2^{64}$.

Extraction. In the random-oracle model, the oracle queries made before C_M is output determine, for every root, the leaves it was built from, unless they contain a collision, which has probability $O(Q^2 2^{-256})$ [67]. A leaf of a shared tree binds the column digests of its members and thus each member's column. A column that cannot be extracted can be opened only by finding a preimage, so we count it as corrupted. This defines every E_j before any challenge.

Setup. Let E_j be not e_j -close. Its rows lie in $\mathbb{F}_p^{n_j} \subset \mathbb{K}_s^{n_j}$. By the correlated-agreement theorem for Reed-Solomon codes in the unique-decoding regime [68, Thm. 1.6], applied to the code over $\mathbb{K}_s$, if a uniform $\psi \in \mathbb{K}_s^{R_j}$ made $\psi^\top E_j$ e_j -close to that code with probability above $n_j/|\mathbb{K}_s|$, then all rows of E_j would agree with codewords over $\mathbb{K}_s$, and hence, by their base coordinates, over $\mathbb{F}_p$, on a common set of $n_j - e_j$ columns, so E_j would be e_j -close. For e_j below a quarter of the distance, Liger's lemma [22] gives the same with $(e_j + 1)/|\mathbb{K}_s| \leq n_j/|\mathbb{K}_s|$. By Fact 1 the s base-field combinations form a uniform ψ , so except with probability n_j/p^s , $\psi^\top E_j$ is e_j -far. The committer's s message rows form one message over $\mathbb{K}_s$, and by Fact 1 the check at column c passes for all of them exactly when its codeword agrees with $\psi^\top E_j$ at c . That happens on at most $n_j - e_j - 1$ columns, a set fixed when the message is sent, so the t_j^0 columns drawn afterwards all fall into it with probability at most $\prod_{i < t_j^0} (n_j - e_j - 1 - i)/(n_j - i)$. The setup proof is a public-coin protocol whose rounds have these errors, so under Fiat-Shamir the bound gains the factor Q [65], [67].

Online. Let every E_j be e_j -close, let $U_j = \text{Enc}(A_j^*)$ be the unique codeword matrix within distance e_j , and let B_j be the at most e_j columns on which $E_j \neq U_j$; C_M fixes B_j . The model M^* computes each weight product $A_\ell^*[X; 1]$ in $\mathbb{F}_p$, lifts it to $(-p/2, p/2)$, and applies the operations of Appendix B.1 to these integers. Let ℓ be the first operation whose claim differs from the value

of M^* ; as before, $\tilde{X}_\ell$ is the input of M^* . The claim is an integer with $|z| < 2^{29} < p/2$ and the value of M^* an integer in $(-p/2, p/2)$, so they differ in $\mathbb{F}_p$ and $D_\ell = Z_\ell - A_\ell^*[\tilde{X}_\ell; 1] \neq 0$; no bound on A^* is needed. In the row layout, if $u_\ell = \chi_\ell^\top A_\ell^*$, Freivalds' check fails except with probability p^{-r} . Otherwise, for a row ρ where they differ, a passing column c satisfies $\text{Enc}(u_{\ell,\rho})_c = (\chi_\ell^\top E_j)_{\rho,c}$; outside B_j the right side equals $\text{Enc}((\chi_\ell^\top A_\ell^*)_{\rho,c})$, so c lies in B_j or in the at most $k_j - 1$ positions where the two codewords agree. The columns are drawn after u_ℓ , which gives the stated term. In the transposed layout the same holds for every w_i , uniform or not, because $w_i^\top E'_j$ agrees with $w_i^\top U'_j = \text{Enc}(A^* w_i)$ outside B_j . A lookup table is the set of rows its root binds, and M^* uses those rows. The round-by-round argument of Appendix B.2 carries over with k_j replaced by $k_j + e_j$. $\square$

The split commitment. Let row ρ of E_j equal $\text{Enc}(A_{j,\rho})$ on a half H of the columns and $\text{Enc}(A'_{j,\rho}) \neq \text{Enc}(A_{j,\rho})$ on the other half. A codeword within distance e_j of this row agrees with $\text{Enc}(A_{j,\rho})$ on at least $n_j/2 - e_j$ columns of H , and with $\text{Enc}(A'_{j,\rho})$ on at least $n_j/2 - e_j$ columns outside H ; it differs from one of the two, so $n_j/2 - e_j \leq k_j - 1$. For $e_j < n_j/2 - k_j + 1$ the split commitment is therefore not e_j -close and the setup proof rejects it except with probability ε_0 . For larger radii it may be close to $\text{Enc}(A_j)$; then M^* uses A_j , and the per-query term with $k_j + e_j$ bounds the acceptance of answers computed with A'_j .

Appendix C. Soundness of V1

This appendix proves Theorem 5. It fixes the notation, states the verifier's checks and the GKR protocol precisely, proves four lemmas, and then proves the theorem.

Notation. $p = 15 \cdot 2^{27} + 1$, $\mathbb{K} = \mathbb{F}_p[\xi]/(\xi^8 - 11)$, and $S = \mathbb{F}_p + \xi\mathbb{F}_p$ is the set of elements of $\mathbb{K}$ whose coordinates of degree 2 to 7 vanish, so $|S| = p^2$. Integers are mapped to $\mathbb{F}_p$ modulo p . For a weight operation ℓ , A_ℓ is an integer matrix with N_ℓ rows and $\tilde{X}_\ell = [X_\ell; 1]$ is the input that the verifier derives. For an entry (i, j) of a cut operation, with $i < N_\ell$ and $j < T$, s_{ij} is the sent value, L_{ij} and W_{ij} are the start and width of its window ($\text{lo}_\ell(s_{ij})$ and $W_\ell(s_{ij})$ for a normal entry, z and 1 for an exception with value z), $\delta_{ij} = (A_\ell \tilde{X}_\ell)_{ij} - L_{ij} \in \mathbb{Z}$ is its residue and $v_{ij} = \delta_{ij} + \xi W_{ij} \in \mathbb{K}$ its leaf. The table is $\mathcal{T} = \{t + \xi w : w \in \mathcal{W}, 0 \leq t < w\}$ and μ holds the multiplicities. An instance γ stacks the rows of its operations, operation ℓ at row offset o_ℓ , in R_γ rows; it has $n_\gamma = n_{\text{row}} + n_{\text{col}}$ variables with $n_{\text{row}} = \lceil \log_2 R_\gamma \rceil$ and $n_{\text{col}} = \lceil \log_2 T \rceil$. Leaf index $\iota = \text{row} \cdot 2^{n_{\text{col}}} + \text{col}$ is *real* if $\text{row} < R_\gamma$ and $\text{col} < T$. Bit vectors are indexed from the least significant bit, $\text{eq}(a, b) = \prod_i (a_i b_i + (1 - a_i)(1 - b_i))$, and the multilinear extension of a function g on $\{0, 1\}^n$ is $\tilde{g}(\rho) = \sum_y \text{eq}(\rho, y) g(y)$. For an integer N , $I_N(r) = \sum_{y < N} \text{eq}(r, y)$, which the verifier computes from the binary digits of N in $O(\log N)$ operations.

The verifier's checks. The verifier rejects at the first failed check. (D1) Ranges: $a \in [-127, 127]$, $\Delta \in R_\Delta =$

$[\text{shift}_\ell(1 - 2^{29}), \text{shift}_\ell(2^{29} - 1)]$, $|z| < 2^{29}$ for clear claims, and a bound on the inputs of cut operations (defence in depth only). (D2) Derivation: every operation without weights is recomputed; the consumer of a cut operation reads s_ℓ . (D3) The token rule on the claims of the LM head. (D4) Exceptions: each listed entry belongs to a cut requantisation, the indices increase strictly within an operation, $|z| < 2^{29}$ and $\text{rq}_\ell(z) = a_{ij}$. (D5) Multiplicities: $|\mathcal{T}|$ entries, $\mu_\tau < p$ for each, and $\sum_\tau \mu_\tau = N_{\text{leaf}}$ in the integers. (G) Every round check of every instance's GKR protocol, and $d_0^\gamma \neq 0$. (F1) Freivalds' equation for the clear operations. (F2) In K_{pre} : $\chi_s^\top y_\ell = u_s \bar{x}_\ell$ for every cut operation and secret vector. (F3) For every instance, $\hat{n} = I_\gamma$ and $\hat{d} = \alpha I_\gamma + 1 - I_\gamma - \tilde{V}_\gamma$, where $I_\gamma = I_{R_\gamma}(r_{\text{row}}) I_T(r_{\text{col}})$, $\tilde{V}_\gamma = \sum_{\ell \in \gamma} (S_\ell - \chi_\ell^\top L_\ell e + \xi \chi_\ell^\top W_\ell e)$, and S_ℓ is $u_\ell \bar{x}_\ell$ for a cut operation in the row layout and $\chi_\ell^\top y_\ell$ for one in the transposed layout or in K_{pre} . (F4) $\sum_\gamma n_0^\gamma / d_0^\gamma = \sum_\tau \mu_\tau / (\alpha - \tau)$, compared by cross-multiplication. (C) The column check of every committed matrix, with the eight coordinate planes of χ_ℓ and u_ℓ (cut, row layout), of $\bar{x}$ and y (cut, transposed layout), or the operands of v_0 (clear).

The GKR protocol of one instance. Layer $n = n_\gamma$ holds the leaves, $(n_\iota, d_\iota) = (1, \alpha - v_\iota)$ for real ι and $(0, 1)$ otherwise. Node y of layer $k < n$ combines the children $2y$ and $2y + 1$:

$$\begin{aligned} n_k(y) &= n_{k+1}(2y) d_{k+1}(2y+1) + n_{k+1}(2y+1) d_{k+1}(2y), \\ d_k(y) &= d_{k+1}(2y) d_{k+1}(2y+1). \end{aligned}$$

(i) The prover sends $n_1(0), n_1(1), d_1(0), d_1(1)$. The verifier computes the root (n_0, d_0) by the same rule, draws θ_0 , and sets the claims $\bar{n} = n_1(0) + \theta_0(n_1(1) - n_1(0))$ and $\bar{d}$ likewise, at the point $\rho = (\theta_0)$. (ii) For $k = 1, \dots, n - 1$ the verifier draws η_k . The claim $c = \bar{n} + \eta_k \bar{d}$ should equal $\sum_{y \in \{0,1\}^k} \text{eq}(\rho, y) F_k(y)$, where $F_k(y) = n_{k+1}(2y) d_{k+1}(2y+1) + n_{k+1}(2y+1) d_{k+1}(2y) + \eta_k d_{k+1}(2y) d_{k+1}(2y+1)$. A sum-check over the k bits of y follows. In round j the prover sends $g_j(0), g_j(2), g_j(3)$ of a polynomial of degree at most 3; the verifier sets $g_j(1) = c - g_j(0)$, draws $r_{k,j}$, and replaces c by $g_j(r_{k,j})$. With $r' = (r_{k,0}, \dots, r_{k,k-1})$, the prover then sends $\tilde{n}_{k+1}(b, r')$ and $\tilde{d}_{k+1}(b, r')$ for $b \in \{0, 1\}$, where b is the lowest bit, and the verifier checks $\text{eq}(\rho, r') [\tilde{n}_{k+1}(0, r') \tilde{d}_{k+1}(1, r') + \tilde{n}_{k+1}(1, r') \tilde{d}_{k+1}(0, r') + \eta_k \tilde{d}_{k+1}(0, r') \tilde{d}_{k+1}(1, r')] = c$. It draws θ_k , sets $\bar{n} = \tilde{n}_{k+1}(0, r') + \theta_k(\tilde{n}_{k+1}(1, r') - \tilde{n}_{k+1}(0, r'))$ and $\bar{d}$ likewise, and moves to the point $\rho \leftarrow (\theta_k, r')$. (iii) The output is the root, the point $\rho = (r_{\text{col}}, r_{\text{row}})$, whose n_{col} lowest coordinates form r_{col} , and the final claims $\hat{n} = \bar{n}$, $\hat{d} = \bar{d}$. The transcript has $4 + \sum_{k=1}^{n-1} (3k + 4)$ elements of $\mathbb{K}$.

Proof of Lemma 1. Let c_v be the number of indices i with $v_i = v$, so that $\sum_i 1/(X - v_i) = \sum_v c_v/(X - v)$, where c_v is read in $\mathbb{F}_p$. The functions $1/(X - v)$ for distinct v are linearly independent over $\mathbb{K}$ (partial fractions are unique), so the identity gives $c_v \equiv 0 \pmod{p}$ for $v \notin \mathcal{T}$ and $c_\tau \equiv \mu_\tau \pmod{p}$ for $\tau \in \mathcal{T}$. Since $c_\tau \geq 0$ and $0 \leq \mu_\tau < p$, the second congruence gives $c_\tau \geq \mu_\tau$. Hence $\sum_{\tau \in \mathcal{T}} c_\tau \geq \sum_\tau \mu_\tau = N = \sum_v c_v$, and $c_v = 0$ for every $v \notin \mathcal{T}$. $\square$

Lemma 4 (windows). *For an integer $m_\ell > 0$, $\text{shift}_\ell(z) = v$ if and only if $\log_\ell(v) \leq z < \log_\ell(v+1)$, and $W_\ell(v) \in \{\lfloor 2^{30}/m_\ell \rfloor, \lceil 2^{30}/m_\ell \rceil\}$. If (D1) and (D4) pass and every residue of a cut operation ℓ lies in $[0, W_{ij})$, then s_ℓ equals $\text{rq}_\ell(A_\ell \hat{X}_\ell)$ for a requantisation and $\text{shift}_\ell(A_\ell \hat{X}_\ell)$ for a residual operation.*

Proof. $\text{shift}_\ell(z) = v$ means $v 2^{30} \leq m_\ell z + 2^{29} < (v+1) 2^{30}$, that is, $(v 2^{30} - 2^{29})/m_\ell \leq z < ((v+1) 2^{30} - 2^{29})/m_\ell$. For an integer z this is $\log_\ell(v) \leq z < \log_\ell(v+1)$. The two bounds differ by $2^{30}/m_\ell$, so their ceilings differ by $\lfloor 2^{30}/m_\ell \rfloor$ or $\lceil 2^{30}/m_\ell \rceil$. Let $Z = (A_\ell \hat{X}_\ell)_{ij}$. For a normal entry, $\delta_{ij} \in [0, W_{ij})$ gives $\text{shift}_\ell(Z) = s_{ij}$; for a residual operation this is the claim, and for a requantisation $|s_{ij}| \leq 127$ by (D1), so $\text{rq}_\ell(Z) = s_{ij}$. For an exception, $W_{ij} = 1$ and $\delta_{ij} = 0$ give $Z = z$, and (D4) checked $\text{rq}_\ell(z) = a_{ij}$. $\square$

Lemma 5 (no wrap-around). *Let $\ell \in \mathcal{C}$ have the honest input, and let (D1) and (D4) pass. If $\delta_{ij} \notin [0, W_{ij})$, then $v_{ij} \notin \mathcal{T}$.*

Proof. By (P4), $|(A_\ell \hat{X}_\ell)_{ij}| < 2^{29}$. We show $|L_{ij}| < 2^{29} + 2^{16}$. For a normal entry of a requantisation, $|s_{ij}| \leq 127$ (by (D1), consistent with the clamp that (P1) requires) and $2^{30}/m_\ell \leq 2^{16}$ by (P3), so $|\log_\ell(s_{ij})| \leq 127.5 \cdot 2^{16} + 1 < 2^{23}$. For a residual operation, $\log_\ell$ is non-decreasing and $\log_\ell(\text{shift}_\ell(z)) \leq z < \log_\ell(\text{shift}_\ell(z)) + W_\ell(\text{shift}_\ell(z))$ by Lemma 4, so $s_{ij} \in R_\Delta$ gives $1 - 2^{29} - 2^{16} < \log_\ell(s_{ij}) \leq 2^{29} - 1$. For an exception, $|z| < 2^{29}$ by (D4). Hence $|\delta_{ij}| < 2^{30} + 2^{16}$. If $\delta_{ij} < 0$, then $\delta_{ij} \bmod p = p + \delta_{ij} > p - 2^{30} - 2^{16} = 939,458,561 > 2^{16} \geq W_{ij}$. If $\delta_{ij} \geq W_{ij}$, then $\delta_{ij} < 2^{30} + 2^{16} < p$, so $\delta_{ij} \bmod p = \delta_{ij} \geq W_{ij}$. In both cases $\delta_{ij} \bmod p \notin [0, W_{ij})$. Since 1 and ξ are linearly independent over $\mathbb{F}_p$, an element $t + xw$ of $\mathcal{T}$ equals v_{ij} only if $w = W_{ij}$ and $t = \delta_{ij} \bmod p$, which would require $\delta_{ij} \bmod p < W_{ij}$. $\square$

Lemma 6 (fractional-sum GKR). *Fix α and the true leaves of instance γ , and let n_k^*, d_k^* be the true layers. If the root (n_0, d_0) computed in step (i) differs from (n_0^*, d_0^*) , then, except with probability $\varepsilon_{\text{GKR}}(n_\gamma) = (1 + \sum_{k=1}^{n_\gamma-1} (3k+2))/p^8$ over the verifier's challenges, either a check (G) fails or $(\hat{n}, \hat{d}) \neq (\tilde{n}_{n_\gamma}^*(\rho), \tilde{d}_{n_\gamma}^*(\rho))$.*

Proof. We show that each step keeps a false claim false except with the stated probability. In step (i) the root is a function of the four sent values, so at least one of them differs from its true value. The sent and the true values define lines in θ_0 , one pair for the numerators and one for the denominators, and at least one pair is distinct; two distinct lines meet at most once, so the claims at θ_0 are false except with probability $1/p^8$. In layer k , if $(\bar{n}, \bar{d}) \neq (\tilde{n}_k^*(\rho), \tilde{d}_k^*(\rho))$, then c differs from $\tilde{n}_k^*(\rho) + \eta_k \tilde{d}_k^*(\rho)$ except with probability $1/p^8$, since a nonzero affine function of η_k has at most one root. This true value equals $\sum_y \text{eq}(\rho, y) F_k^*(y)$ by the definition of the tree and of the multilinear extension. The summand, written with $\tilde{n}_{k+1}^*(0, y)$, $\tilde{n}_{k+1}^*(1, y)$ and the corresponding denominators, has degree at most 3 in each variable, so by the soundness of the sum-check [69], except with probability $3k/p^8$ either a round fails or the final c

differs from $\text{eq}(\rho, r') F_k^*(r')$, the value of the summand at r' with the true children. If the four sent child values were the true ones, the verifier's check would compare c with exactly this value and fail; so at least one is false, and as in step (i) the new claims at (θ_k, r') are false except with probability $1/p^8$. Each layer k thus adds $(3k+2)/p^8$ and step (i) adds $1/p^8$. After layer $n_\gamma - 1$ the claims refer to the leaf layer, which gives the statement. $\square$

Lemma 7 (final values). *With e and χ_ℓ as in Section 5, the true leaf layer of instance γ satisfies $\tilde{n}^*(\rho) = I_\gamma$ and $\tilde{d}^*(\rho) = \alpha I_\gamma + 1 - I_\gamma - \sum_{\ell \in \gamma} \chi_\ell^\top (A_\ell \hat{X}_\ell - L_\ell + \xi W_\ell) e$, where $I_\gamma = I_{R_\gamma}(r_{\text{row}}) I_T(r_{\text{col}})$.*

Proof. The numerators are 1 on real leaves and 0 elsewhere, so $\tilde{n}^*(\rho) = \sum_{\ell \text{ real}} \text{eq}(\rho, \ell)$. Since eq factorises over the column and row bits and the real leaves form the product of the first T columns and the first R_γ rows, this sum is $I_T(r_{\text{col}}) I_{R_\gamma}(r_{\text{row}})$. The denominators are $\alpha - v_\ell$ on real leaves and 1 elsewhere, and $\sum_{\ell} \text{eq}(\rho, \ell) = 1$, so $\tilde{d}^*(\rho) = \alpha I_\gamma - \sum_{\ell \text{ real}} \text{eq}(\rho, \ell) v_\ell + (1 - I_\gamma)$. The real leaf in row $o_\ell + i$ and column j is $v_{ij} = (A_\ell \hat{X}_\ell - L_\ell + \xi W_\ell)_{ij}$ in $\mathbb{K}$, and its weight is $\text{eq}(r_{\text{row}}, o_\ell + i) \text{eq}(r_{\text{col}}, j) = (\chi_\ell)_i e_j$, which gives the sum. $\square$

Proof of Theorem 5. We bound the probability that every check passes, so we may assume that the deterministic checks (D1) to (D5) pass. We follow the order of the checks.

Step 1: the first wrong operation. Call a clear operation locally correct if its claims equal $A_\ell \hat{X}_\ell$, and a cut operation locally correct if every residue lies in its window, where $\hat{X}_\ell$ is the input that the verifier derives. If every operation is locally correct, then by Lemma 4 each consumer of a cut operation reads exactly the value of the honest computation on the same input, and by induction over the graph in topological order every derived tensor equals the honest one. The LM head is clear, so its claims are the honest logits, $y = M(x)$, and the token rule holds for them. Hence some operation is locally incorrect. Let ℓ^* be the first in graph order; its derived input is the honest input.

Step 2: a clear ℓ^ .* The argument of Theorem 3 applies to its committed matrix $j \in \mathcal{J}_\mathcal{O}$. The range check keeps the wrong claim distinct from the true value modulo p . With the true folded rows, Freivalds' check fails except with probability p^{-r} ; with others, the column check fails except with probability $\prod_{i < t_j} (k_j - 1 - i)/(n_j - i)$, unless the prover finds a hash collision. In $\mathcal{K}_{\text{pre}}$ the bound is p^{-r} .

Step 3: a cut ℓ^ gives an off-table leaf.* Some residue of ℓ^* lies outside its window, so by Lemma 5 its leaf is not in $\mathcal{T}$. The multiset of true leaves of all cut operations is determined by C_M , x and the first message, which fixes the derived inputs, the sent values and the exceptions. The first message also fixes μ . Both are fixed before α is drawn.

Step 4: the logUp identity fails at α . Since (D5) passed, Lemma 1 shows that $\Phi(X) = \sum_i 1/(X - v_i) - \sum_\tau \mu_\tau/(X - \tau)$ is a nonzero rational function. Every leaf and every table element has zero coordinates of degree 2 to 7, so the poles of Φ form a set $P \subseteq S$ with $|P| \leq N_{\text{leaf}} + |\mathcal{T}|$, and $\Phi(X) \prod_{w \in P} (X - w)$ is a nonzero polynomial of degree

below $|P|$. The challenge α is uniform on $\mathbb{K} \setminus S$, which has $p^8 - p^2$ elements and contains no pole, so $\Phi(\alpha) = 0$ with probability at most $(N_{\text{leaf}} + |\mathcal{T}|)/(p^8 - p^2)$. Otherwise $\sum_{\gamma} n_0^{*\gamma}/d_0^{*\gamma} = \sum_i 1/(\alpha - v_i) \neq \sum_{\tau} \mu_{\tau}/(\alpha - \tau)$, where every true root denominator is a product of nonzero factors $\alpha - v_i$. If (F4) passes, with $d_0^{\gamma} \neq 0$ by (G), some instance γ has $n_0^{\gamma}/d_0^{\gamma} \neq n_0^{*\gamma}/d_0^{*\gamma}$, so its root is not the true root.

Step 5: the error reaches the final claims. By Lemma 6, except with probability $\varepsilon_{\text{GKR}}(n_{\gamma})$, either (G) fails or the final claims of γ differ from the true values at ρ .

Step 6: the final check. By Lemma 7, if $S_{\ell} = \chi_{\ell}^{\top} A_{\ell} \hat{X}_{\ell} e$ for every $\ell \in \gamma$, the right-hand sides of (F3) are the true values, and (F3) rejects. Otherwise some S_{ℓ} is wrong, so $u_{\ell} \neq \chi_{\ell}^{\top} A_{\ell}$ (row layout) or $y \neq A\bar{x}$ (transposed layout, or K_{pre}). In mode C with the row layout, write $\chi_{\ell} = \sum_{q < 8} \xi^q \chi^{(q)}$ and $u_{\ell} = \sum_{q < 8} \xi^q u^{(q)}$, where $\chi^{(q)}$ and $u^{(q)}$ have entries in $\mathbb{F}_p$; then $u^{(q)} \neq \chi^{(q)\top} A_{\ell}$ for some q . Because C_M is honest, every row of the encoded matrix $\hat{A}$ is a codeword, so $\chi^{(q)\top} \hat{A} = \text{Enc}(\chi^{(q)\top} A_{\ell})$ and $\text{Enc}(u^{(q)})$ are distinct codewords that agree on at most $k_j - 1$ of the n_j positions. The t_j distinct columns are drawn after u_{ℓ} is sent, so they all fall on agreeing positions with probability at most $\prod_{i < t_j} (k_j - 1 - i)/(n_j - i)$; otherwise (C) fails, unless the prover finds a hash collision. The transposed layout is symmetric: $\bar{x}^{(q)\top} \hat{A}' = \text{Enc}(A\bar{x}^{(q)})$ differs from $\text{Enc}(y^{(q)})$ for some q . In K_{pre} , $y_{\ell} - A_{\ell} \bar{x}_{\ell}$ has a nonzero coordinate plane $d \in \mathbb{F}_p^{N_{\ell}}$, and $\chi_s^{\top} (y_{\ell} - A_{\ell} \bar{x}_{\ell}) = 0$ requires $\chi_s^{\top} d = 0$, which holds with probability $1/p$ for a uniform secret vector χ_s that the prover has not seen; over the r independent secret vectors this is p^{-r} .

Step 7: union bound. Adding the probabilities of steps 2 and 4 to 6 over the operations, the instances and the committed matrices gives the bound of Theorem 5. $\square$

Reuse in K_{pre} . As in v0, the prover learns each verdict and could repeat an accepted forgery. Until one is accepted, the verdicts do not depend on the secret vectors, since y_{ℓ} , the GKR transcripts and α do not involve them. A false acceptance among Q queries that reuse the vectors therefore occurs with probability at most Q times the bound.

Fiat-Shamir. The interactive protocol is round-by-round sound, as in Section 4.4: once a message makes the statement doomed, only the next challenge can rescue it. For the new rounds this probability is at most $1/p^8$ for θ_k and η_k , $3/p^8 = 2^{-245.7}$ for a sum-check round, and $(N_{\text{leaf}} + |\mathcal{T}|)/(p^8 - p^2) \leq 2^{-215.3}$ for α . All are below the per-round error that the Fiat-Shamir parameters of v0 allow at $\lambda = 128$, so its state-restoration bound for Q oracle queries carries over unchanged.

Remarks. (i) *Saturated entries.* If the prover omits an exception, the entry keeps its normal window, which the true pre-activation misses; if it lists a false value z , the residue is nonzero in a window of width 1. Either way Step 3 applies. (ii) *Residual operations.* Δ is unclamped, so its windows are those of shift_{ℓ} ; the verifier applies the clamp of the sum itself in (D2). (iii) *Input bounds.* The proof does not use the input range check of (D1), because Step 1 works with the honest input of ℓ^* ; the check is defence in depth.

(iv) *The claim bound (P4).* In K_{pre} the verifier evaluates (P4) from its own weights. In mode C it holds no weights, and (P4) is a property of the committed model, checked when the honest commitment is computed, as is v0's claim range. With an untrusted commitment (Section 4.5) the bound must be published with the model. (v) *Completeness.* Honest multiplicities are below $2^{29} < p$ by (P5). The exception bin of width 1 is not covered by (P5), so a proof with more than 2^{29} clamped entries cannot be encoded; this affects completeness only.